

탐색의 피로는 줄이고, 구매의 확신은 높이는 초개인화 AI 셀러

독보적인 가전 구매 경험을 완성하는 L-Pick

5팀 빵껍살마라짜장산물

김다빈 이지연 이지우 인서정 한수민 황병관



목차

1 추진 배경

2 핵심 난제

3 고객 경험 목표

4 데이터 분석 과정

5 타겟 고객

6 핵심 고객 경험

7 핵심 경험 컨셉

8 서비스 기능

9 Key Accelerator

10 Performance Tracker

SWOT 분석

Strength

- LG AI 전담 조직(AI 연구원, LG CNS 등) 협업 기반 AI 기술 및 서비스 구축 역량 e.g. ThinQ on AI
- '가전은 LG다'라는 긍정적 소비자 인식 e.g. 가전 TOM 브랜드
- 글로벌 시장에서의 높은 브랜드 인지도 및 선호도 e.g. 생활가전 글로벌 1위
- 혁신적이고 차별화된 가전 라인업 e.g. 티움, Puricare Pet Water Fountain
- ESG 기반의 친환경 생산 방식 e.g. 희토류 재활용
- 프리미엄 가전 시장에서의 경쟁 우위

Opportunity

- 업계 내 오프라인 운영 비용 절감 및 매출 증대를 위한 D2C 채널 강화 흐름
- 가전 구독 수요 증가에 따른 가전 구독 시장 성장
- 초고령화 사회 진입에 따른 시니어 대상 홈 서비스 수요 급증
- 친환경·고효율 트렌드에 따른 프리미엄 가전 선호도 증가
- 핵가족화 및 1인 가구 증가에 따른 콤팩트 가전 수요 증가
- 콘텐츠 플랫폼 이용 증가에 따른 수요 확대

Weakness

- 제품 정보 및 기능 파악이 어려운 자사물 AI챗봇
- 해상 운송에 의존하는 유통 구조
- 중국 저가형 가전 부상에 따른 가격 경쟁력 약화
- 업계 경쟁 심화로 인한 마케팅 비용 증가
- 핸드폰 사업 부재로 인한 확보 가능한 개인 데이터의 한계 존재
- 가전구독 종료 후 폐가전 처리 비용 발생

Threats

- 경쟁 기업 및 업계 전반의 AI 이커머스 솔루션 고도화
- 중국 가전 시장 성장으로 인한 점유율 하락
- 글로벌 정책 변화로 인한 관세 부담
- 전자제품 폐기 비용 규제 정책
- 핵가구화 및 1인 가구 증가로 인한 가전 대량 구매 및 대형 가전 구매 감소
- 신혼부부 결혼 비용 증가로 인한 가전 구매 예산 감소

SWOT 분석

Strength

- LG AI 전담 조직(AI 연구원, LG CNS 등) 협업 기반 AI 기술 및 서비스 구축 역량 e.g. ThinQ on AI
- '가전은 LG다'라는 긍정적 소비자 인식 e.g. 가전 TOM 브랜드
- 글로벌 시장에서의 높은 브랜드 인지도 및 선호도 e.g. 생활가전 글로벌 1위
- 혁신적이고 차별화된 가전 라인업 e.g. 티움, Puricare Pet Water Fountain
- ESG 기반의 친환경 생산 방식 e.g. 히트류 재활용
- 프리미엄 가전 시장에서의 경쟁 우위

LG AI 전담 조직(LG AI 연구원, LG CNS 등) 협업 기반 AI 기술 및 서비스 구축 역량

[LG 전자 'AI 상담 어시스트']



'2025 콜센터품질지수' 조사에서 6년
연속 가전제품 부문 최우수기업으로 선정

[LG ThinQ ON AI]



AI 기반 IoT 허브 ThinQ ON AI, 누적 1만 세대 보급 돌파

SWOT 분석

Opportunity

- 업계 내 오프라인 운영 비용 절감 및 매출 증대를 위한 D2C 채널 강화 흐름
- 가전 구독 수요 증가에 따른 가전 구독 시장 성장
- 초고령화 사회 진입에 따른 시니어 대상 홈 서비스 수요 급증
- 친환경·고효율 트렌드에 따른 프리미엄 가전 선호도 증가
- 핵가족화 및 1인 가구 증가에 따른 콤팩트 가전 수요 증가
- 콘텐츠 플랫폼 이용 증가에 따른 수요 확대

오프라인 운영 비용 절감 및 매출 증대 목적 D2C 채널 강화 흐름

[최근 2년 'D2C' 키워드 포함 기사 워드클라우드 분석]



> 경쟁력, 쇼핑몰 구축, 매출액, 자사몰, 영업이익, 극대화

SWOT 분석

경쟁 기업 및 업계 전반의 AI 이커머스 솔루션 고도화

[삼성전자]
루비콘 프로젝트



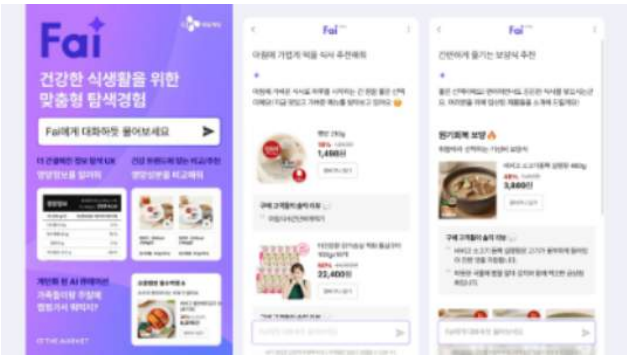
AI 서비스 고도화를 통한
고객 서비스 품질 향상 및
자체 유통 채널 육성 프로젝트 진행

[롯데홈쇼핑]
AI 라이브



롯데 홈쇼핑, AI 모델 도입한
Ai 라이브 방송 결과 가전 주문 3배 증가

[CJ 제일제당]
자사몰 AI 검색 서비스 'Fai'



개인화 상품 추천과 고객 응대 챗봇이
결합된 이커머스 특화 AI 'Fai'

이커머스 솔루션 고도화

- 중국 가전 시장 성장으로 인한 점유율 하락
- 글로벌 정책 변화로 인한 관세 부담
- 전자제품 폐기 비용 규제 정책
- 핵가구화 및 1인 가구 증가로 인한 가전 대량 구매 및 대형 가전 구매 감소
- 신혼부부 결혼 비용 증가로 인한 가전 구매 예산 감소

SWOT 분석 통한 난제 도출

SO 전략

LG AI 기술 역량을 활용해
D2C 채널을 강화해야 함

ST 전략

LG AI 기술 역량을 활용해
치열한 AX 흐름 속 경쟁력을 확보해야 함

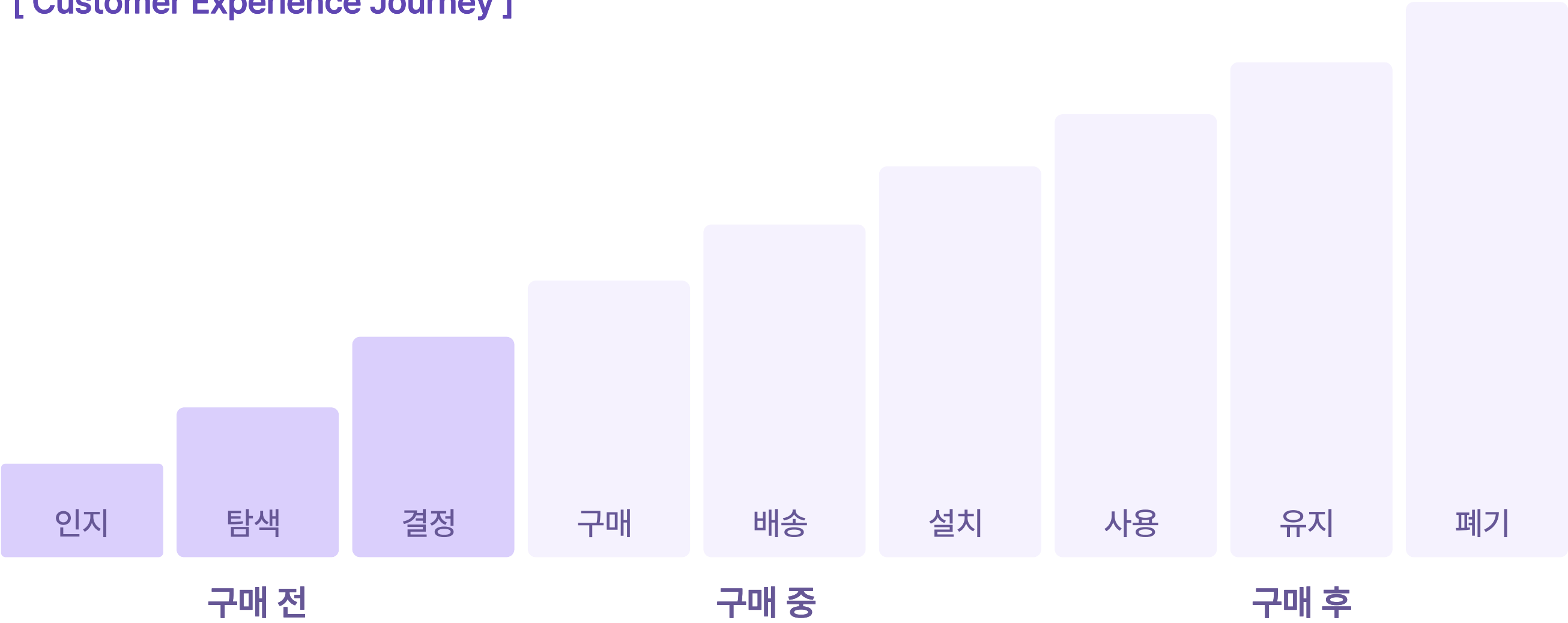


핵심 난제

현재 LG 전자 자사몰의 AI 챗봇은 기초적인 성능에
머물러 있어 D2C 채널 활성화 수준 및 AX 경쟁력이 부족하다.

고객 경험 여정에 따른 세그멘테이션 정의

[Customer Experience Journey]



탐색 단계에서의 행동 방식이 다르면 구매 이후 경험과 만족도가 달라진다.
초기 탐색 채널에서의 만족도가 실제 구매 결정으로 이어지는 강력한 연결고리가 된다.

Lemon, K. N., & Verhoef, P. C. (2016). Understanding Customer Experience Throughout the Customer Journey. Jo

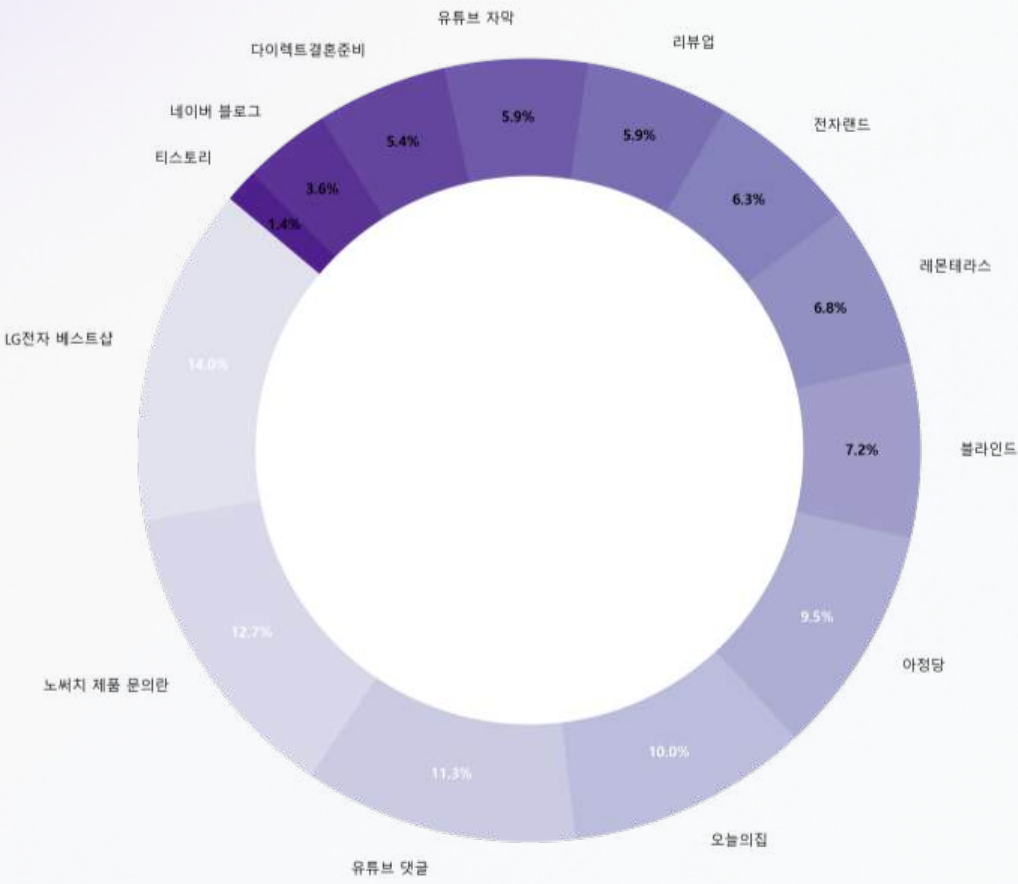
데이터 수집 전략

사이트	목적	키워드	비고
블로그	구매 경험/후기 기반 온오프라인 채널 선택 행동 파악	가전 온라인 구매, 가전 오프라인 구매, 가전 구매 후기, 온오프라인 비교	- 네이버 - 티스토리 2021~2026 설정
카페(네이버)	질문/고민글 기반으로 구매 전 의사결정 파악	가전 추천, 가전 브랜드 비교, AS 좋은 가전, 렌탈, 가전 후기, 에어컨/TV/냉장고 구매, 에어컨/TV/냉장고 후기	- 다이렉트결혼준비 - 레몬테라스 - 리뷰업 - 아정당 2021~2026 설정
유튜브	영상 댓글, 자막 기반 실사용 반응, 비교/추천, 성향 파악	신제품 가전 추천, 가전 업그레이드, 가전 브랜드 비교, 가전 후기	- 유튜브 자막 - 유튜브 댓글 2021~2026 설정
노써치	고관여 소비자, 구매 가이드를 원하는지 파악	-	- 노써치 제품 문의란 2021~2026 설정
오프라인 매장	실제 제품 구매 고객들의 후기 파악	-	- LG전자 베스트샵 - 전자랜드 2021~2026 설정
오늘의집	디자인, 인테리어, 신혼/자취 맥락 소비 분석	가성비 가전, 가전 후기, 가전 만족도, 가전 브랜드 비교	- 오늘의집 2021~2026 설정
블라인드	직장인 맥락에서의 가전 구매 고민/경험 파악	가전 구매, 온라인 구매 후기, 오프라인 구매 후기, 가전 추천	- 블라인드 2021~2026 설정

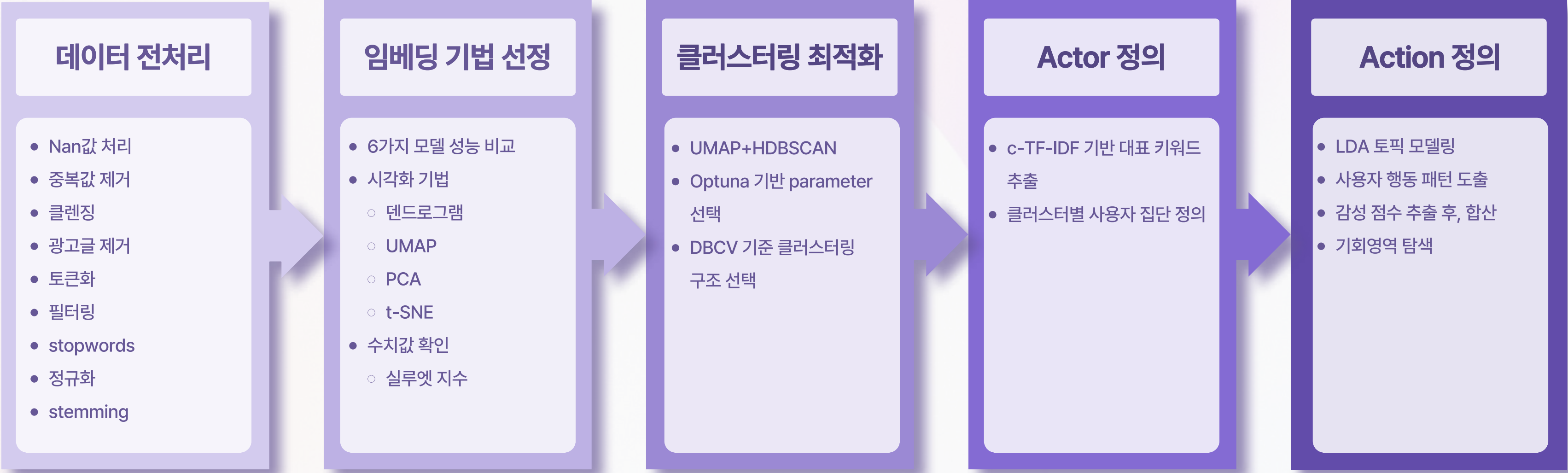
크롤링 전체 데이터: 약 27만건



전처리 후 : 34,180건



전체 데이터 처리 과정

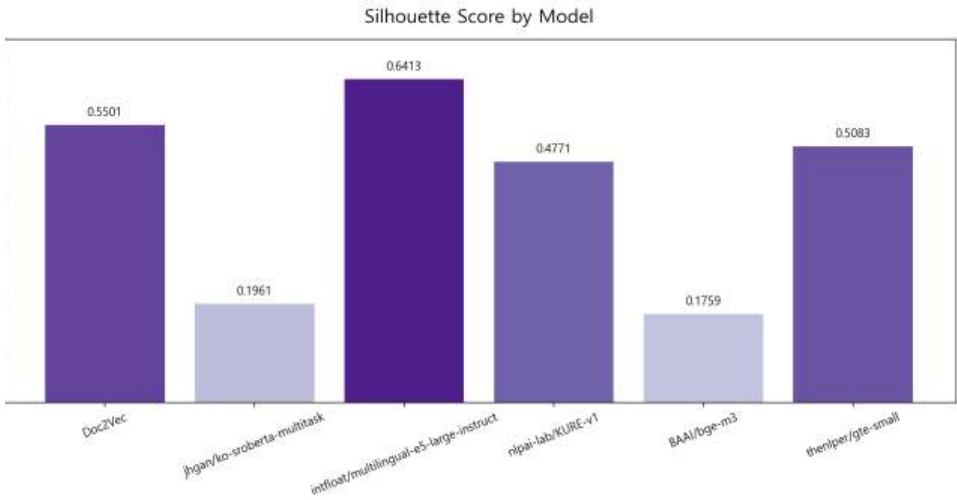


데이터 1차 전처리

Nan 값 처리	중복 제거	클렌징	광고글 제거
 수집된 데이터 중 결측 데이터 식별 후 제거(drop)	 본문의 내용이 전체가 똑같은 시 같은 데이터로 판단하고 해당 데이터 삭제	 - html/url/이메일/멘션 제거 - 해시태그는 #만 제거 - 반복 문자는 한 문자로 처리 - 반복 초성 처리 - 이모지, 특수문자 등 감정 나타내는 경우 보존 - 숫자 뒤 한글이 같이 있을 시 보존 - 의미 없는 자음 모음 제거 - 긴 공백 제거 - 길이 50자 미만 제거	 - hugging face의 pre-training 된 sms-spam-classifier 모델 사용 - 스팸 판단 score가 설정한 thershold 이상일 시 제거 - 수동으로 관련이 없다고 생각되는 글 전체 제거

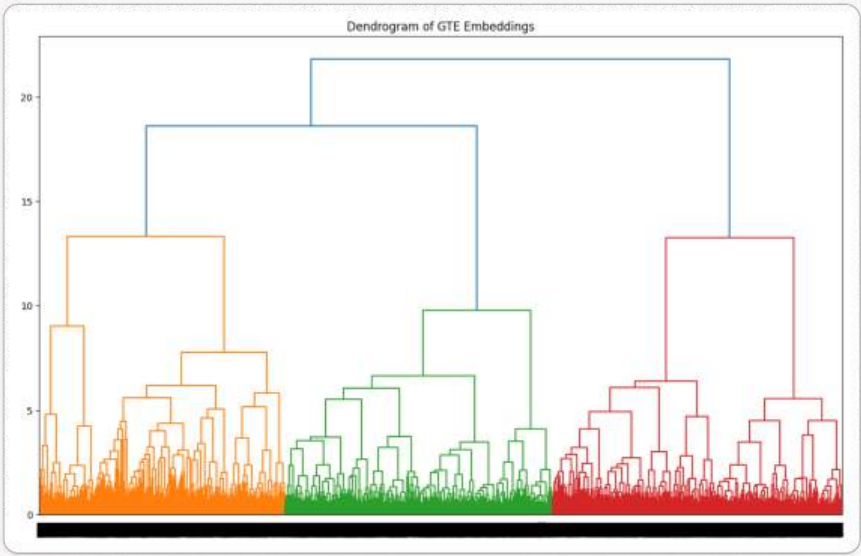
데이터 임베딩 모델

- 대규모 다국어 데이터를 기반으로 학습된 Transformer 기반 문장 임베딩 모델
- contrastive learning을 통해 의미가 유사한 문장은 가까운 벡터 공간에 배치하도록 학습됨
- retrieval 및 semantic similarity benchmark에서 높은 성능을 기록한 고성능 embedding 모델
- 다양한 표현을 사용하는 리뷰·댓글 데이터에서도 의미 기반 유사도를 잘 반영하여 클러스터 구조가 명확하게 형성됨

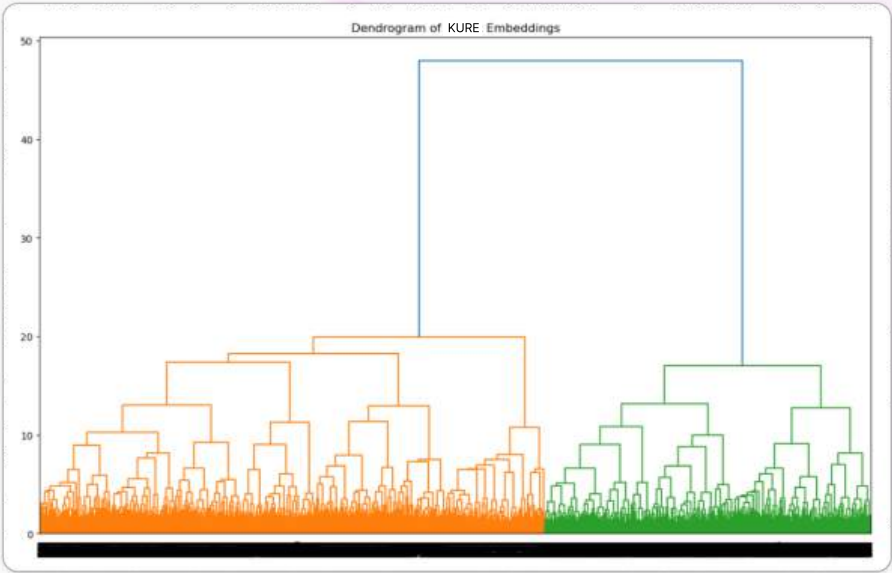


모델	입력 형태	모델 유형	차원	특징	장점	실루엣 지수
Doc2Vec	토큰화 된 문장	전통 임베딩(Word2Vec 기반)	300	문서를 벡터로 학습하는 고전 임베딩 방법	가볍고 CPU에서도 빠름	0.5501
jhgan/ko-sroberta-multitask	문자열	Sentence Transformer	768	한국어 문장 임베딩에 특화된 SBERT	한국어 semantic similarity 성능 좋음	0.1961
intfloat/multilingual-e5-large-instruct	문자열	Transformer embedding	1024	retrieval benchmark에서 매우 높은 성능	semantic similarity 매우 우수	0.6413
nlpai-lab/KURE-v1	문자열	한국어 특화 embedding	768	한국어 retrieval / clustering 성능 강화	한국어 데이터에서 높은 성능	0.4771
BAAI/bge-m3	문자열	Multilingual embedding	1024	dense + sparse + multi-vector 지원	성능 매우 높음, multilingual	0.1759
thenlper/gte-small	문자열	lightweight transformer	384	가벼운 embedding 모델	빠르고 메모리 적음	0.5083

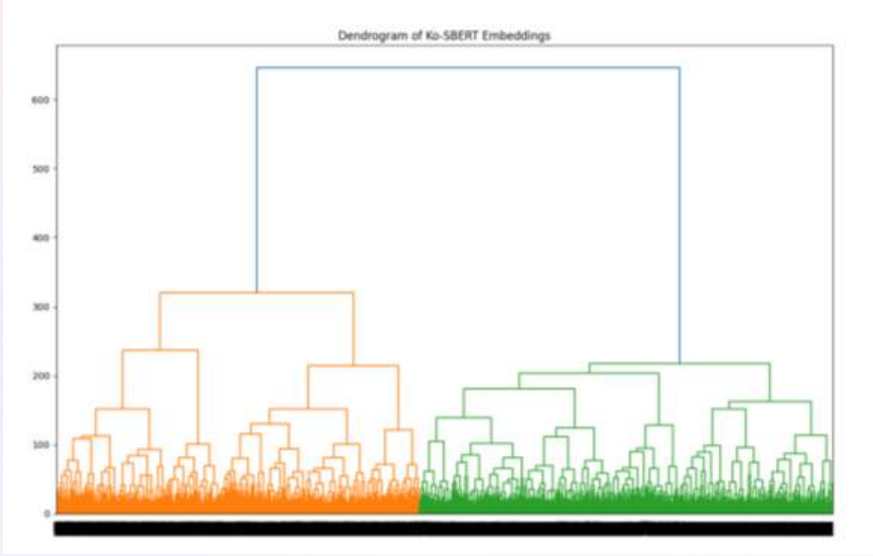
임베딩 모델 별 덴드로그램



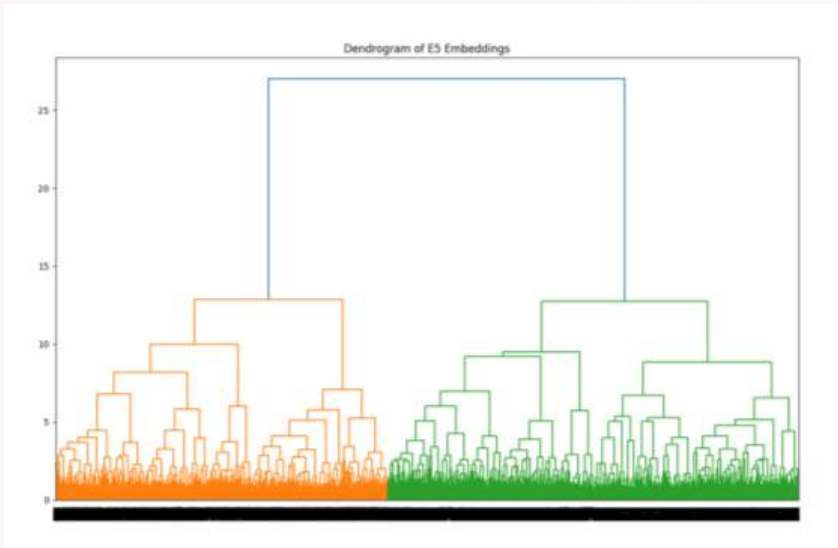
GTE



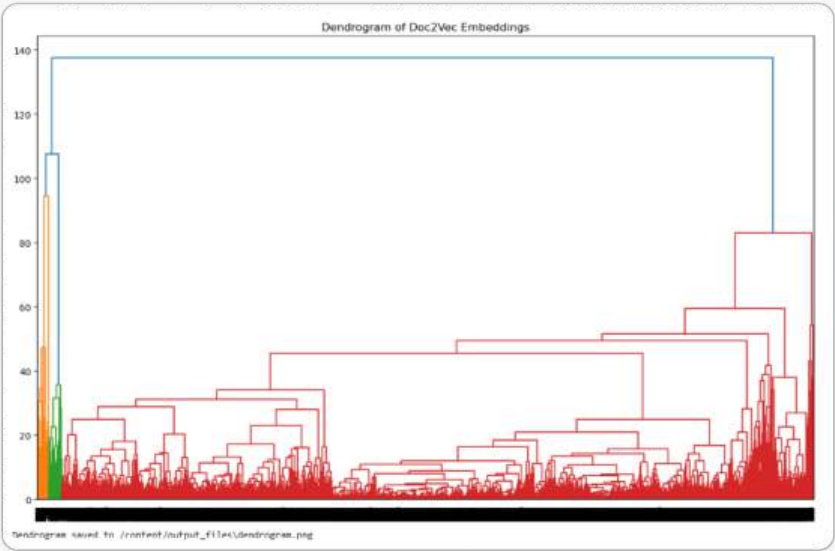
KURE



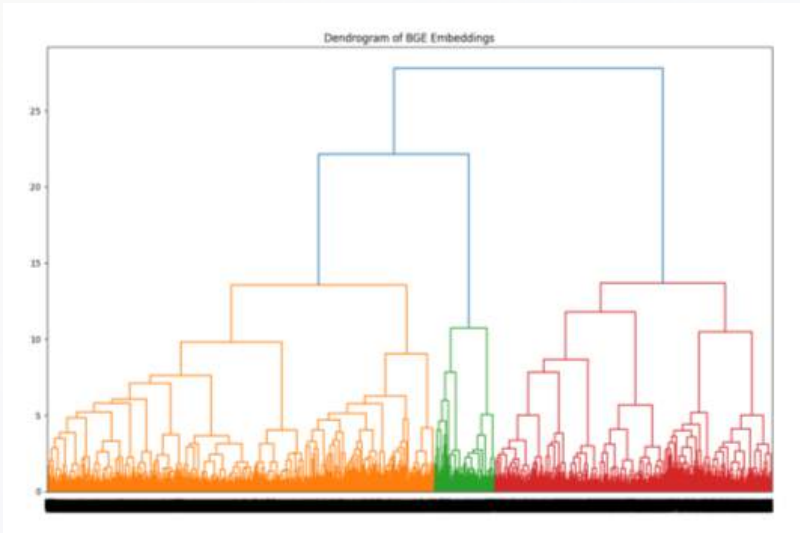
Ko-SBERT



E5

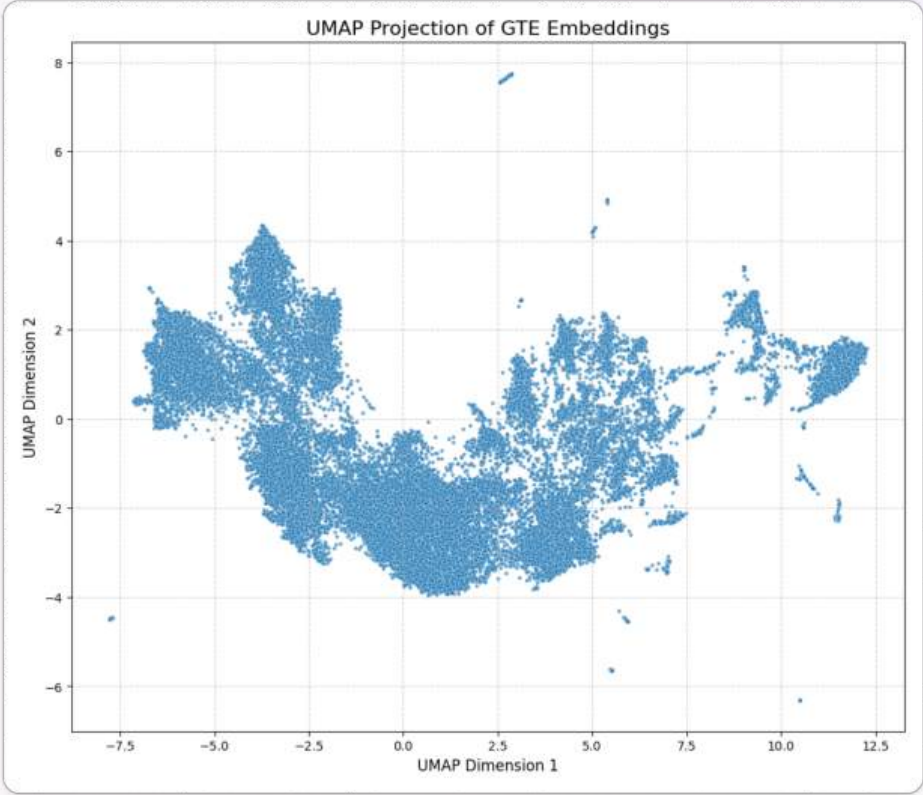


Doc2Vec

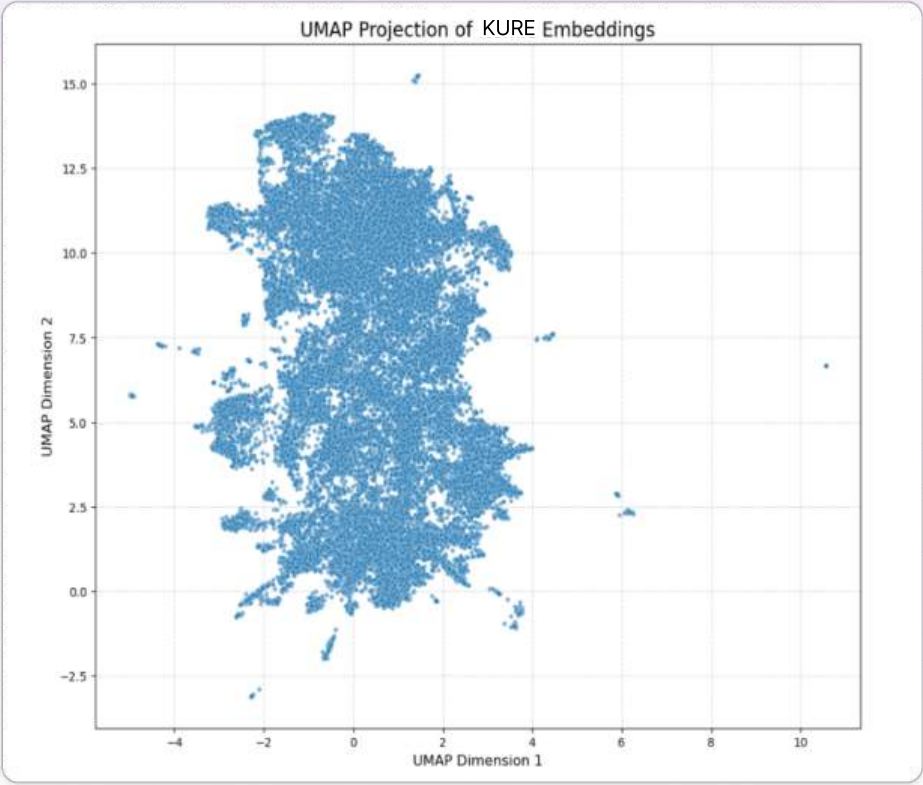


BGE

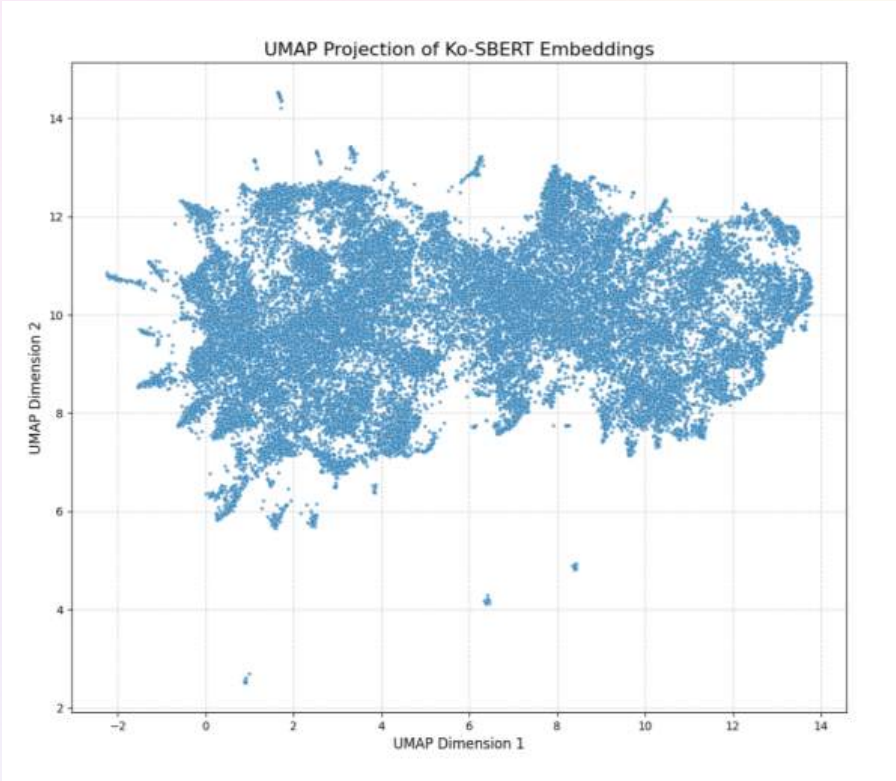
임베딩 모델 별 UMAP



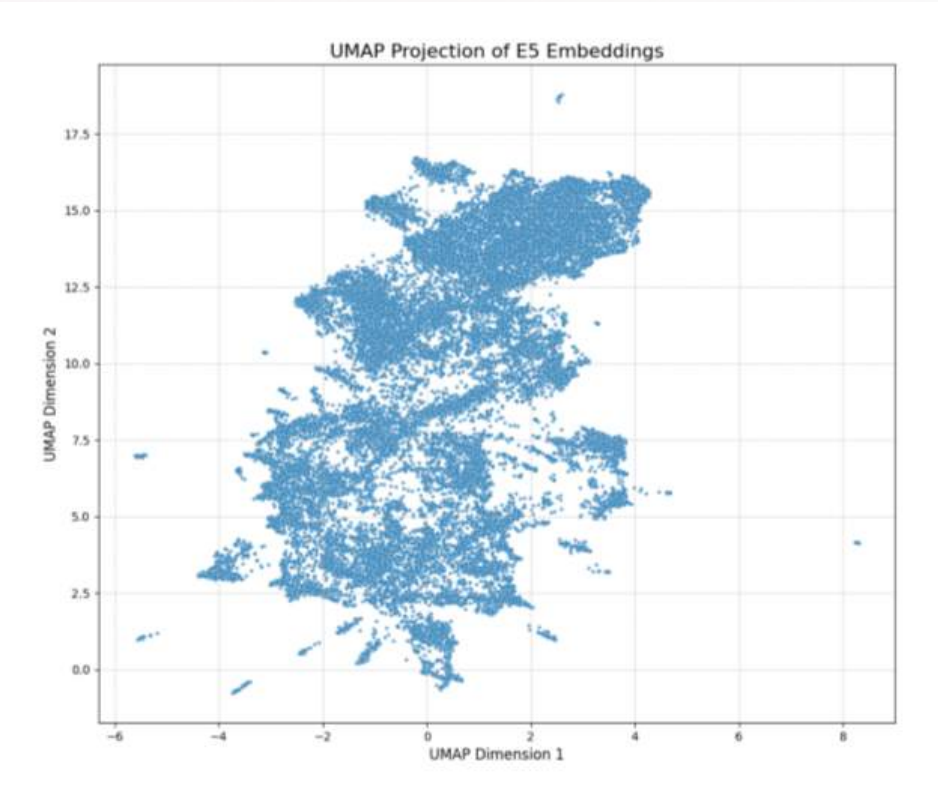
GTE



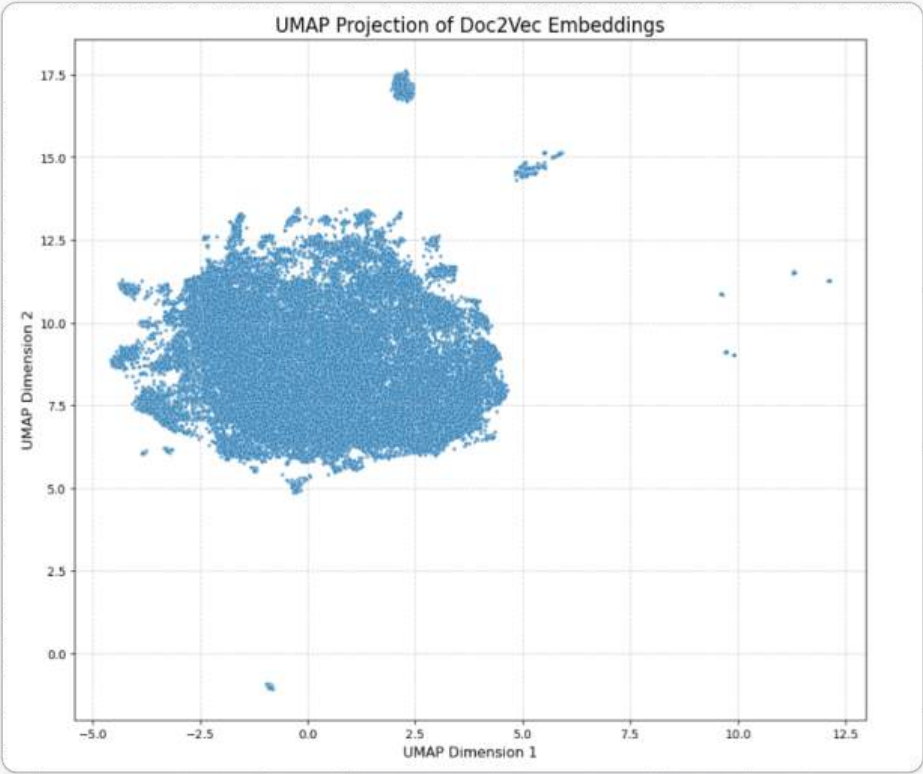
KURE



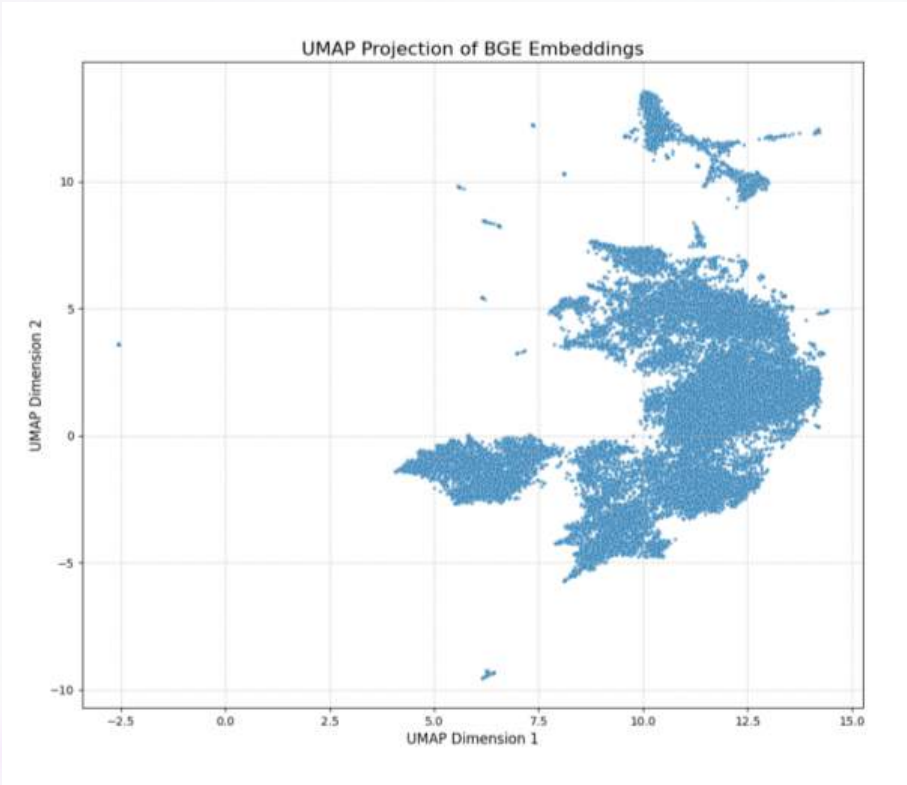
Ko-SBERT



E5

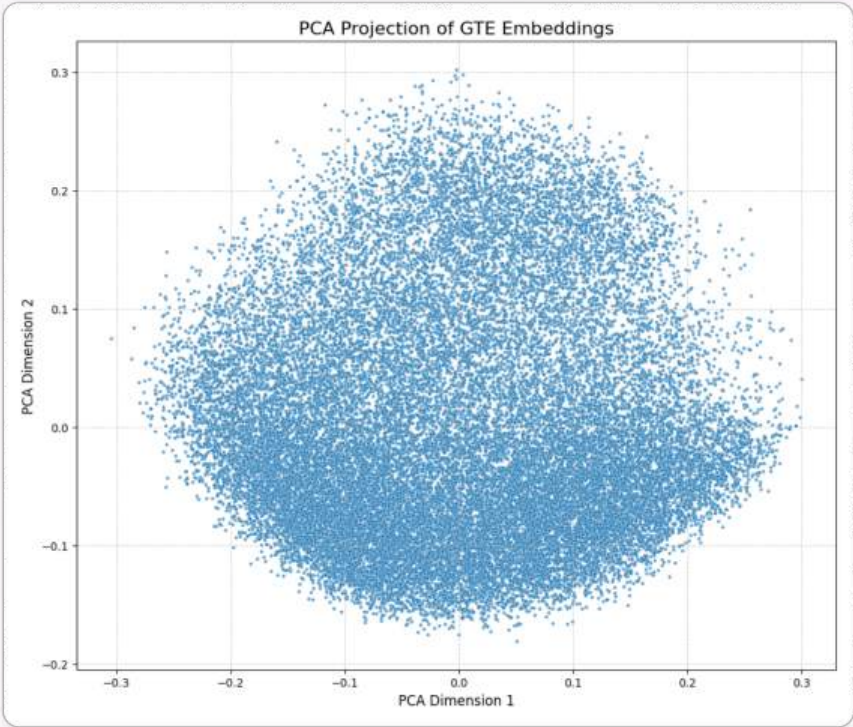


Doc2Vec

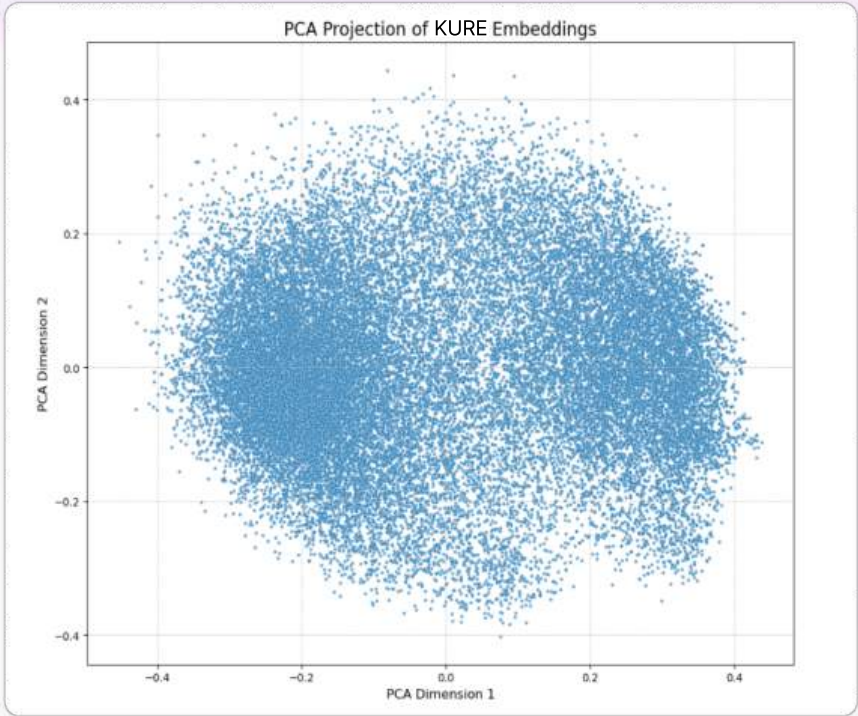


BGE

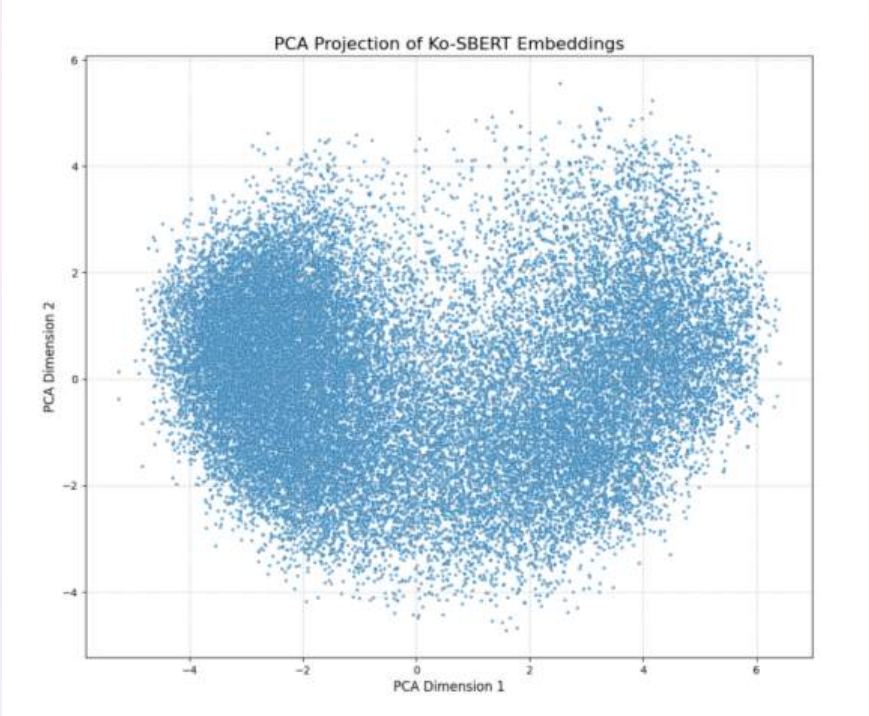
임베딩 모델 별 PCA



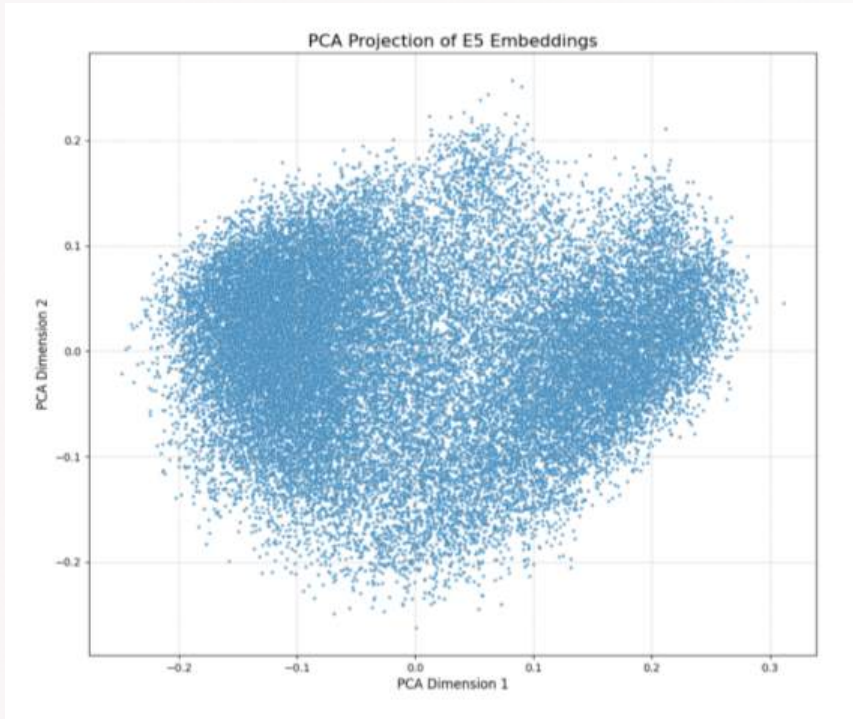
GTE



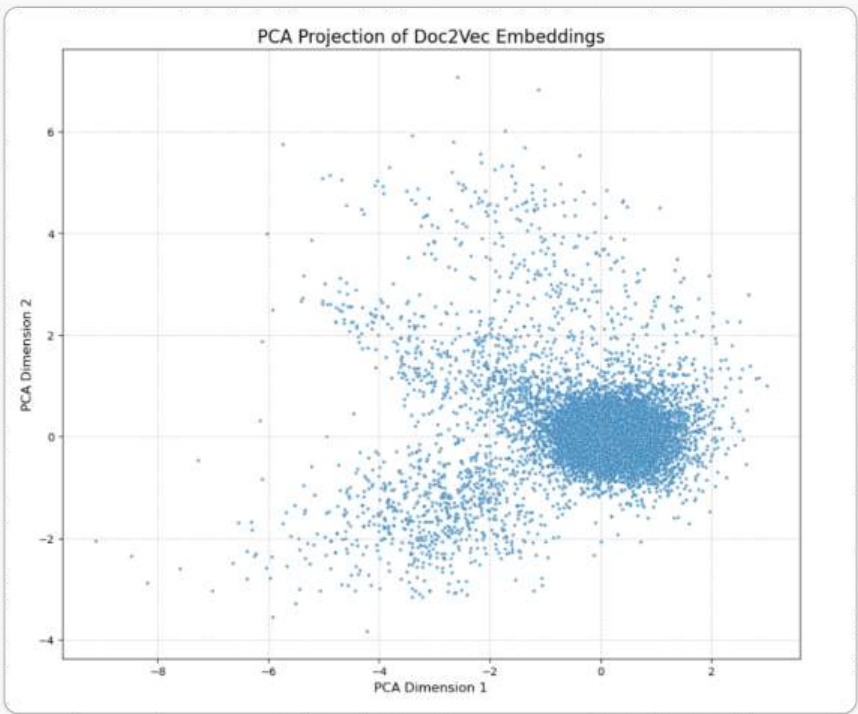
KURE



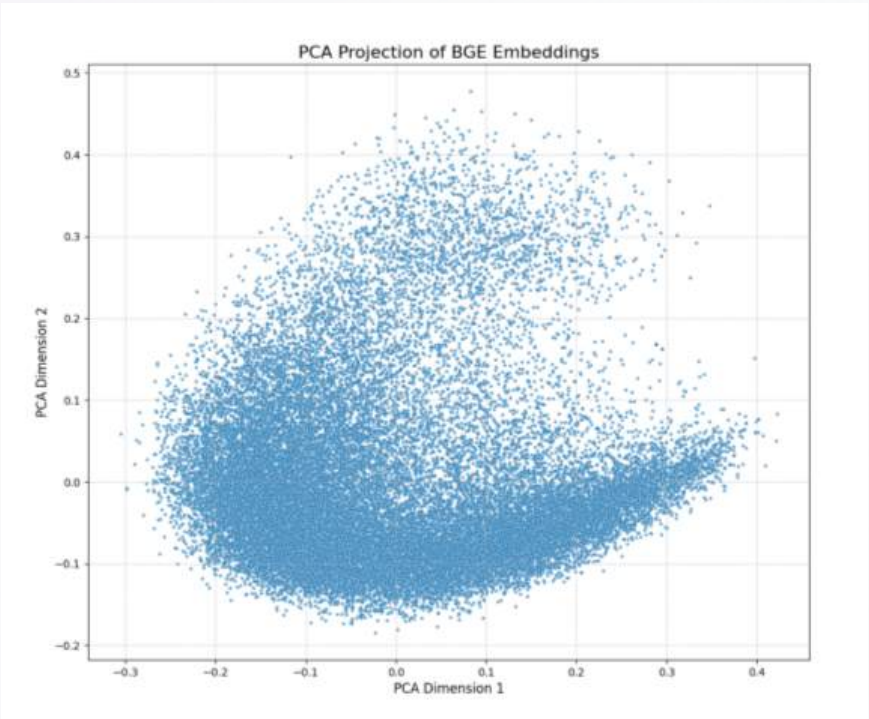
Ko-SBERT



E5

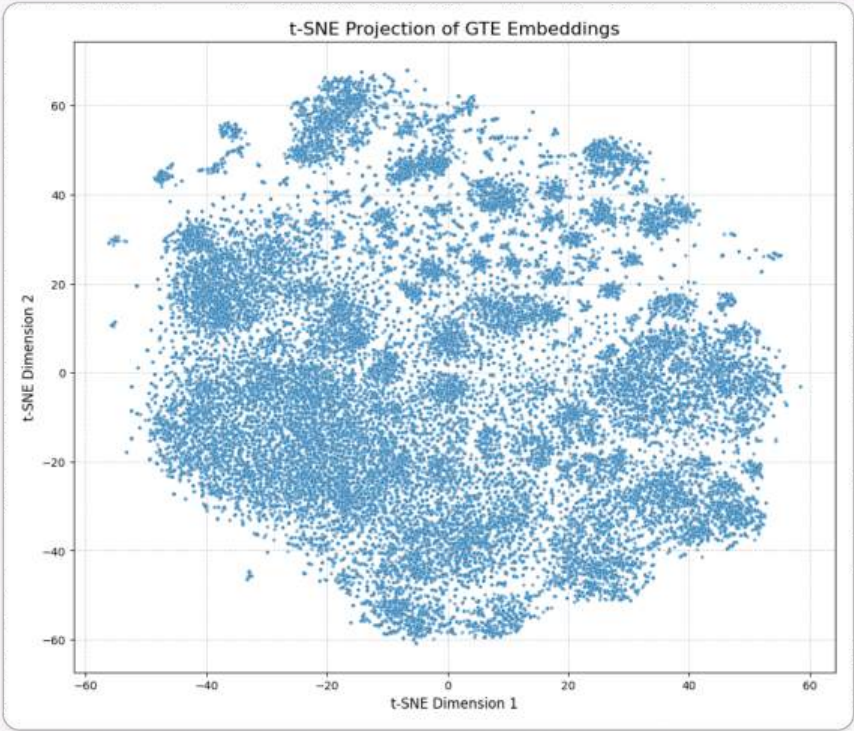


Doc2Vec

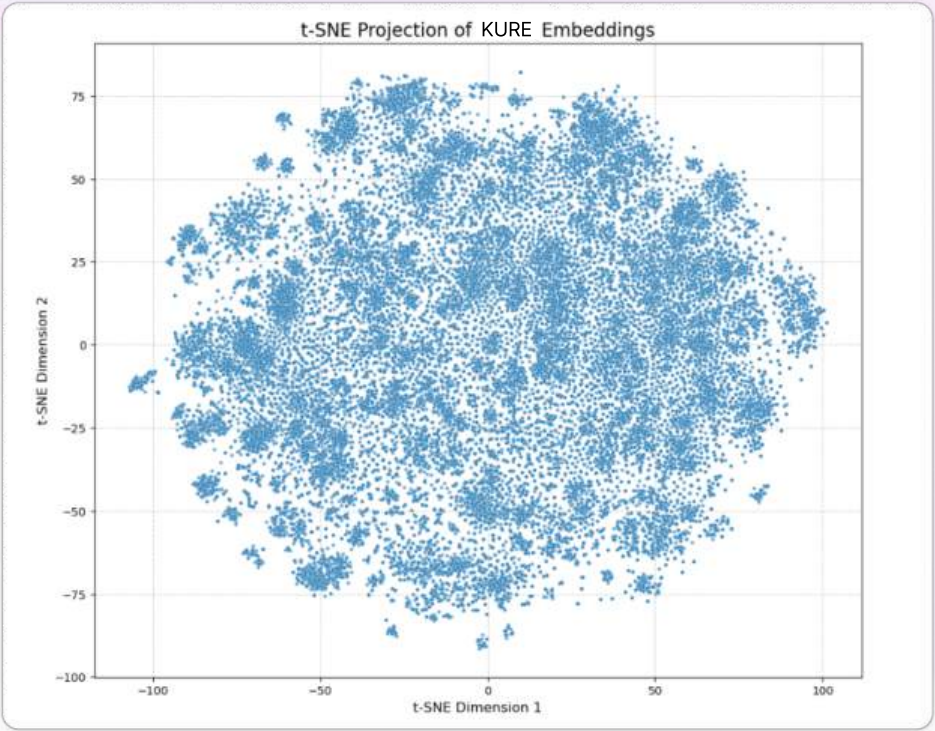


BGE

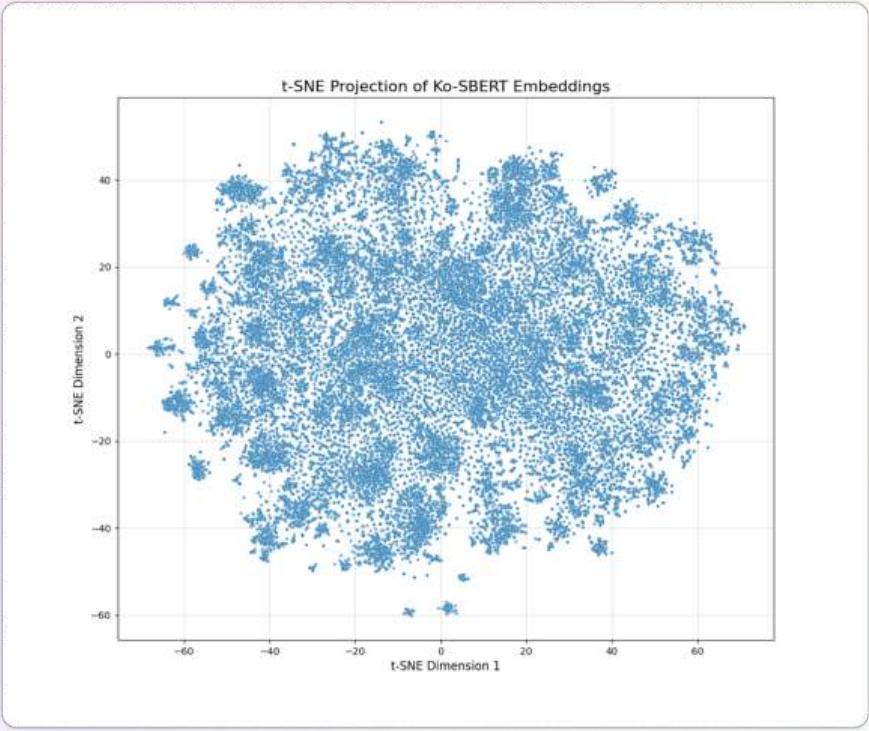
임베딩 모델 별 t-SNE



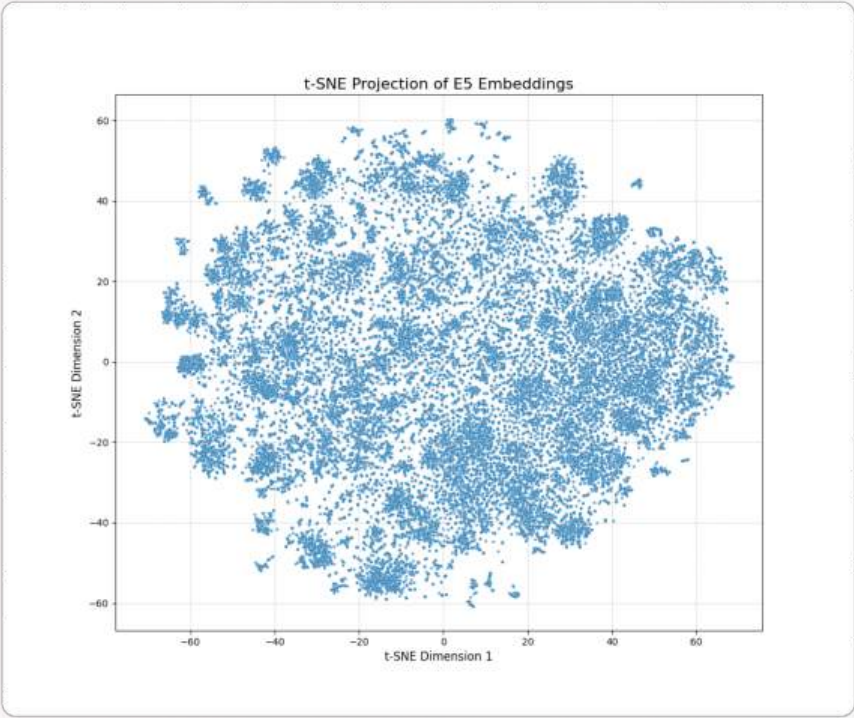
GTE



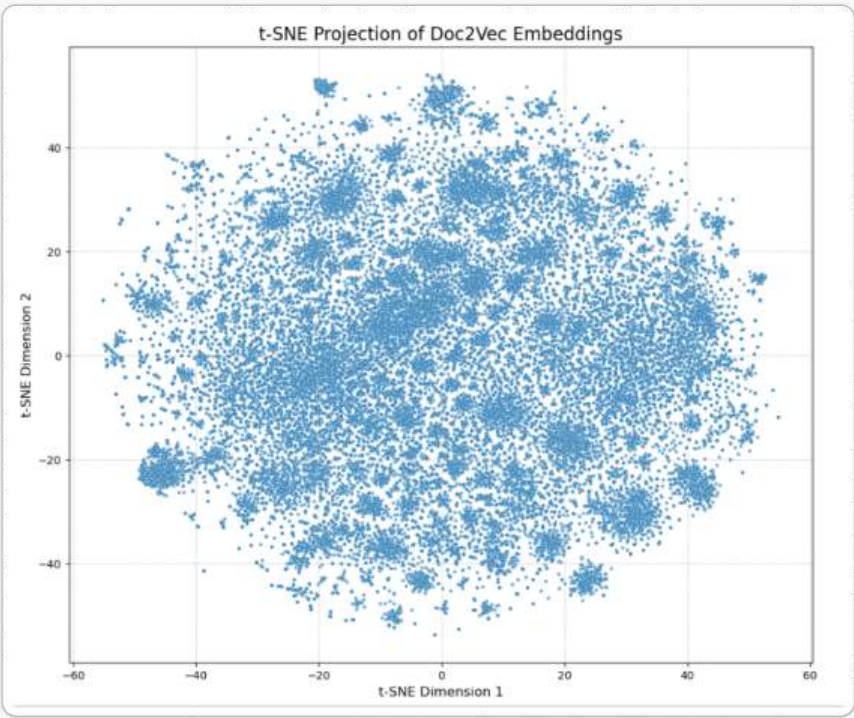
KURE



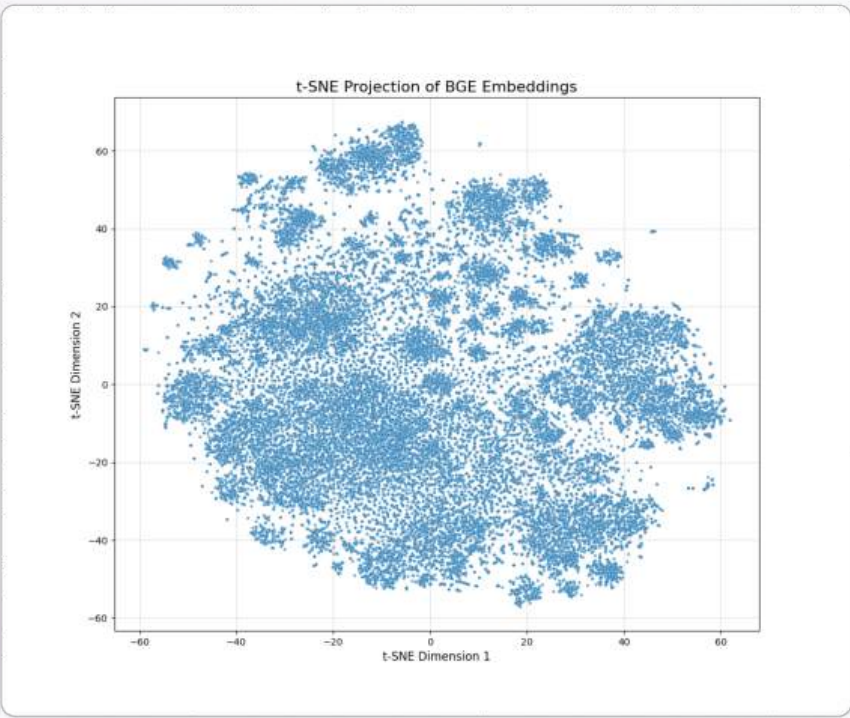
Ko-SBERT



E5

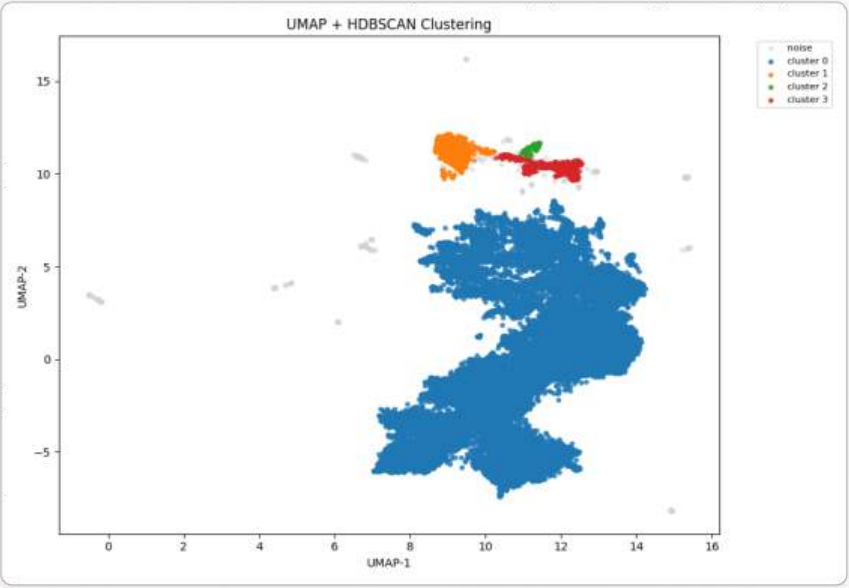


Doc2Vec

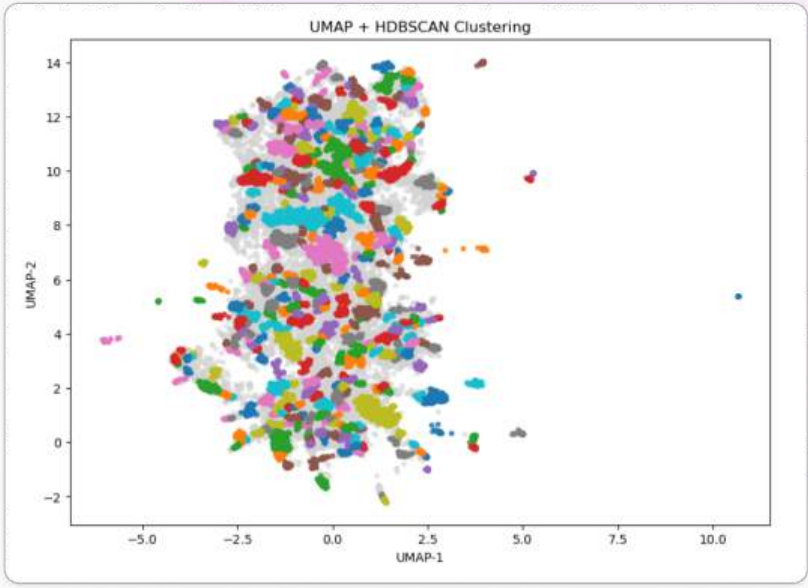


BGE

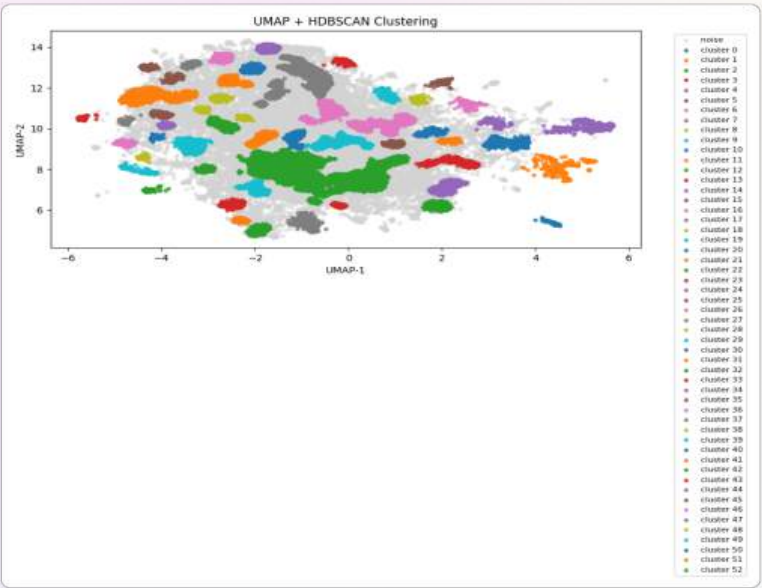
임베딩 모델 별 UMAP + DBSCAN



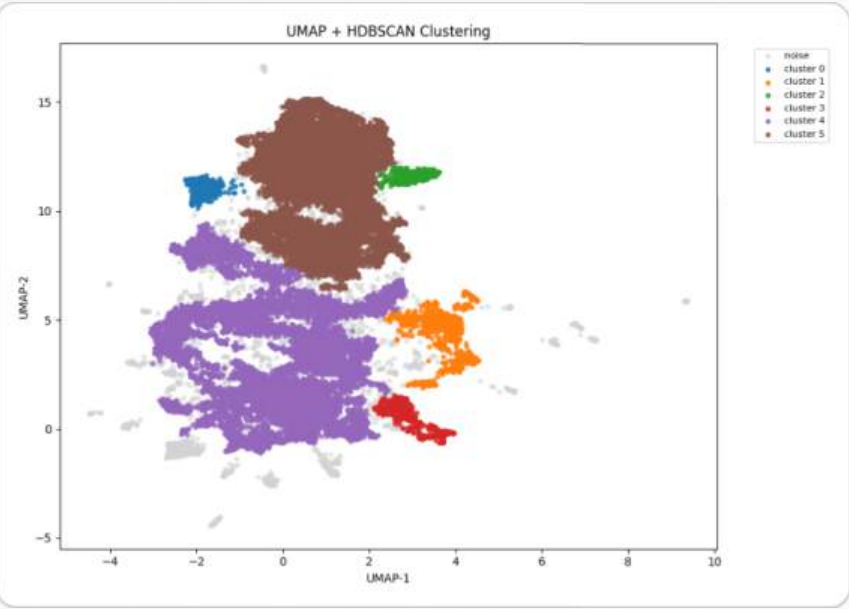
GTE



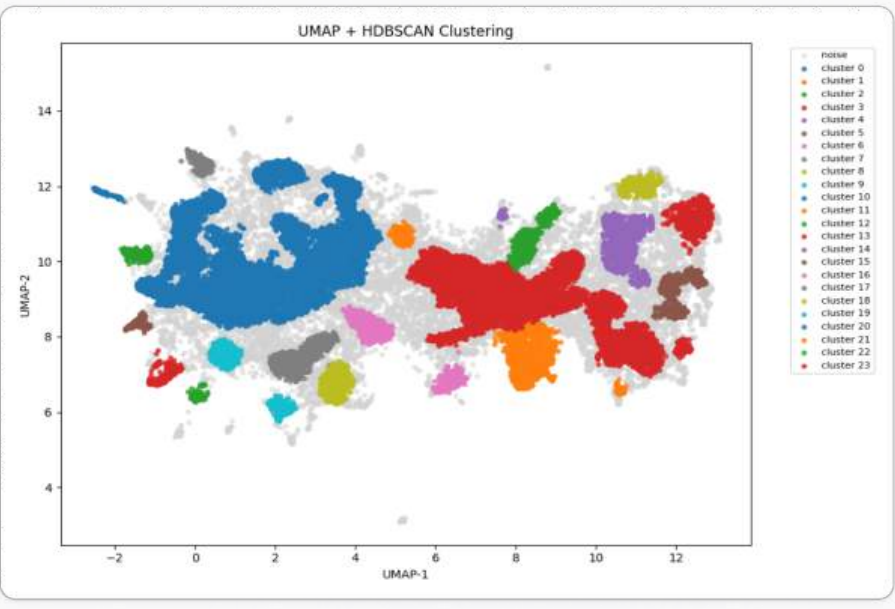
KURE



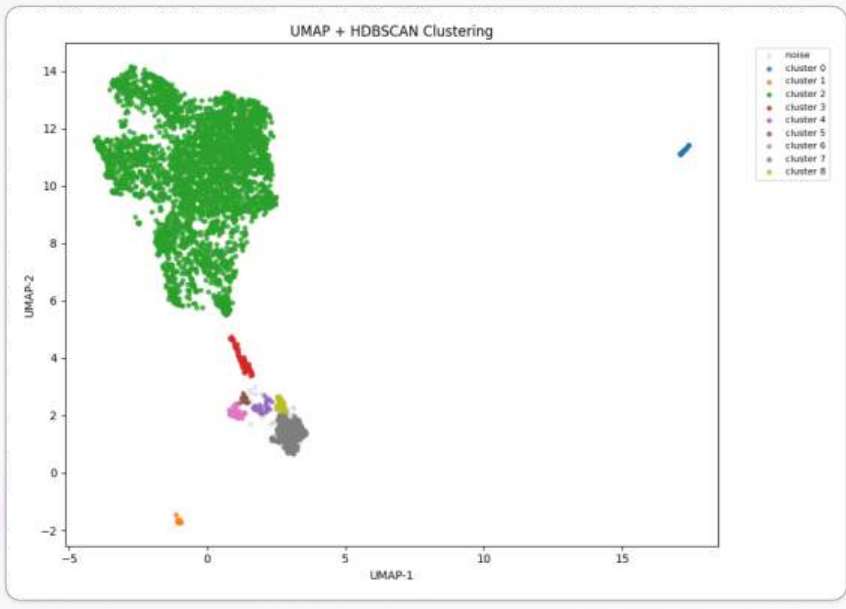
Ko-SBERT



E5



Doc2Vec



BGE

데이터 2차 전처리

토큰화	필터링	StopWords	정규화	Stemming
 • 일반 명사 • 고유 명사 • 동사 • 형용사 • 위의 형태소만 남기고 제거	>  • 기존에 문자열 모델을 학습 시키기 위해 존재했던 반응 표현 (e.g. 초성) 등 추가로 제거 • 한 글자 단어 제거	>  • 총 332개의 불용어를 txt 파일을 이용해 제거 • 추가적으로 제거 되지 않은 단어를 수동으로 100개 가량 추가하여 제거	>  • 띄어쓰기가 되어 있어 떨어진 단어 결합 • 줄임말, 초성, 유행어 사전을 찾아 해당 단어를 기본 형태로 바꾸는 작업 진행	>  • 동사/형용사 기본형 변환 • 단어 활용형 통일 (e.g. 했다 -> 하다) • 모델 입력 일관성 확보

분석 프로세스

01

UMAP

- 고차원 임베딩 벡터를 저차원 공간으로 축소
- 데이터 구조 보존
- HDBSCAN 군집 형성 보조

02

HDBSCAN

- 밀도 기반 클러스터링
- 데이터 분포 기반 군집 형성
- 노이즈 데이터 자동 분

03

클러스터 품질 평가

- DBCV 지표 사용
- 군집 내부 밀도 및 군집 간 분리도 평가

04

최적 파라미터 선정

- Optuna 기반 자동 탐색
- UMAP/HDBSCAN 파라미터 최적화
- UMAP의 $n_neighbors : 15$ / $min_dist : 0.76$ / $n_components : 14$
- HDBSCAN의 $min_cluster_size : 398$ / $min_samples : 39$

05

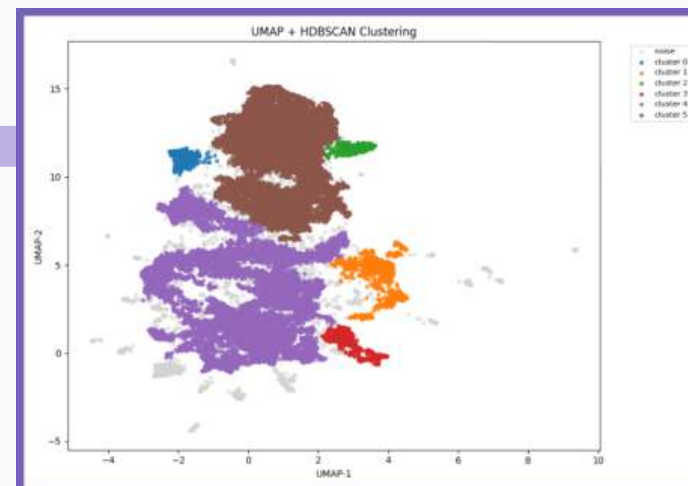
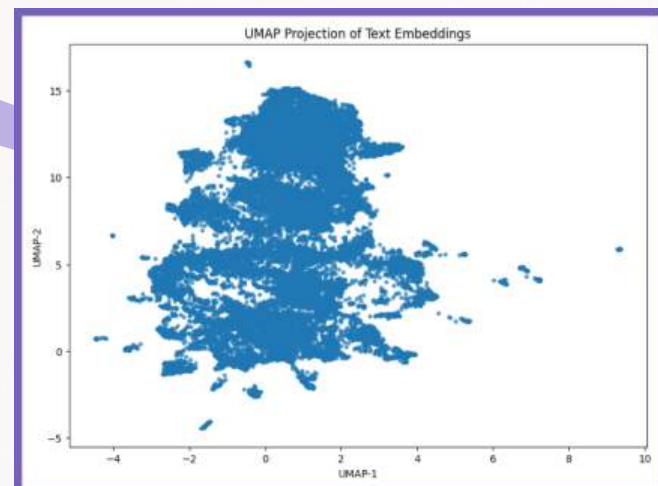
최적의 군집 구조 결정

- DBCV 최댓값 기준
- 최적 클러스터 구조 설정
- 시각화를 통해 6개로 최적의 클러스터 확정

06

노이즈 제거

- Cluster = -1 데이터 제거
- 1628개 제거



분석 프로세스

07

UMAP

- HDBSCAN에서 생성된 cluster label 사용
- 각 문서는 특정 cluster에 속함

08

CountVectorizer (n-gram)

- 단어 출현 빈도 계산
- unigram / bigram 적용
- 의미 단위 유지

09

c-TF-IDF 계산

- cluster 내부 빈도 (TF)
- 다른 cluster 대비 희소성 (IDF)
- cluster 대표 단어 중요도 계산

11

Actor 해석

10

클러스터 대표 키워드 추출

- 대표 키워드를 기반으로 사용자 집단 성격 정의

12

최종 LDA 모델 학습

- 선택된 토픽 수로 모델 학습
- 문서별 토픽 분포 계산
- 토픽 키워드 추출

13

노이즈 제거

- 토픽 키워드 기반 행동
- 패턴 정의

Actor

Actor 0



24.16%

실리 탐색형

넘치는 정보 속에서 진정으로 가치있는
정보를 찾고싶은 고객

"제품이 너무 많은데 내 집에 맞는걸
어떻게 골라야 할까?"
"광고성 리뷰 말고 사람이 쓴 후기는 없나?"

#제품정보 #리뷰

Actor 1



28.77%

불만검증형

제품에 대한 사용자의 불만을 공유받고자
하고, 스펙을 정밀하게 비교하는 고객

"나만 겪는 문제일까?"
"실제로 사용하면 소음이 크지 않을까?"

#추천 #디자인 #성능

Actor 2



16.06%

채널별 조건비교형

온/오프라인 구매 조건을 꼼꼼하게
비교해 결정하는 고객

"저 매장보다 이 매장이 혜택이 더 좋네?"
"온라인이 가격이 더 싸잖아?"

#견적 #혜택

Actor

Actor 3



20.85%

온오프라인 발품형

온오프라인에서 발품을 팔아서라도 저렴한
가격에 좋은 제품 및 서비스를 받고 싶은 고객

"같은 제품을 저렴하게 사고 싶은데
어디서 구매해야 할까?"
"배송까지 좋은 서비스를 가지려면..."

#발품 #가격비교

Actor 4



5.4%

가성비 혼수 탐색형

신혼가전 혜택을 받아 좋은 제품을
가성비 있게 구매하고 싶은 고객

"사야 하는게 많은데...
좋은 제품을 어디서 구매해야 할까?"
"제품에 대해 정보를 알려줬으면 좋겠다~"

#신혼 #다품목 #혜택

Actor 5



4.76%

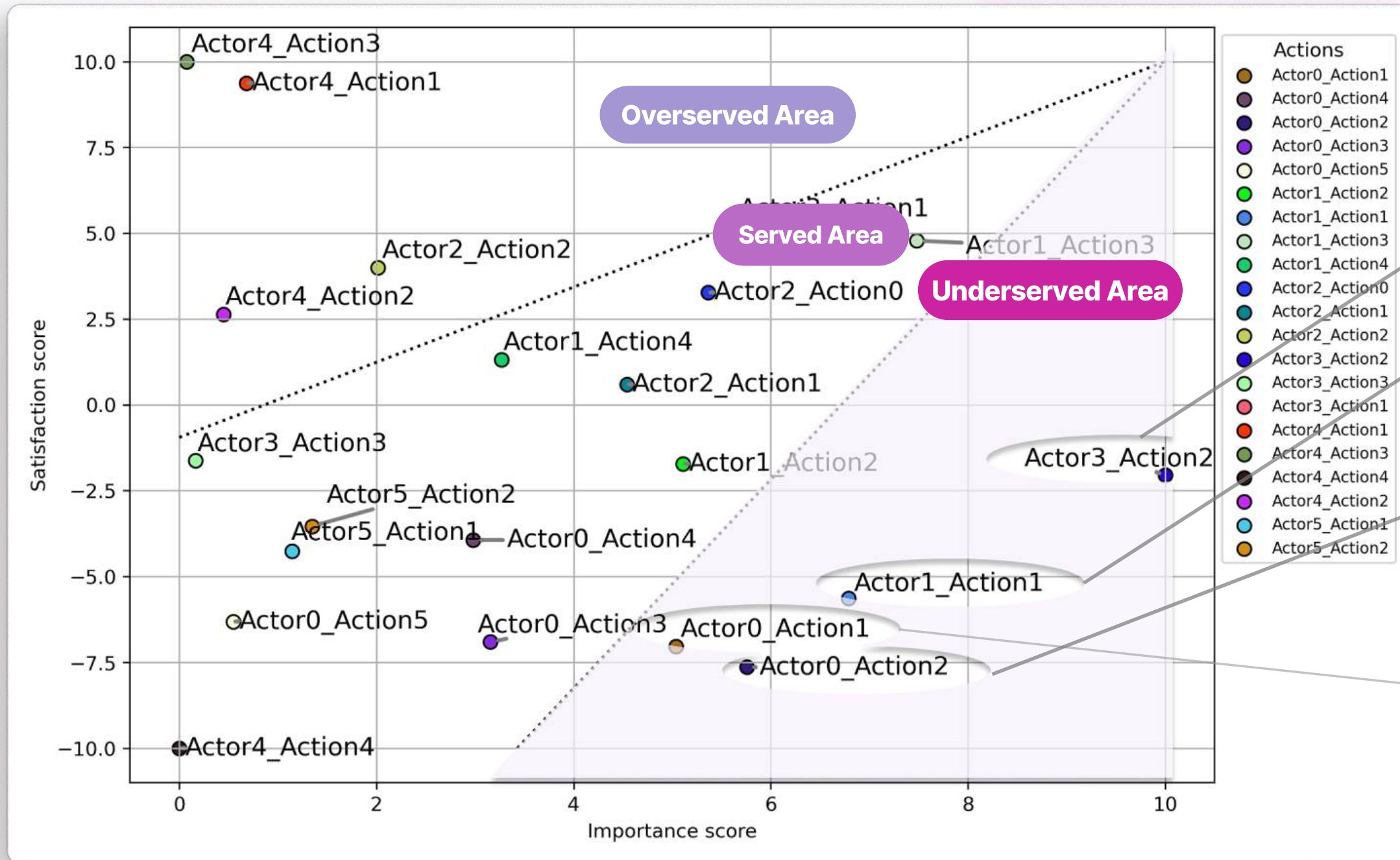
브랜드별 성능 비교형

브랜드별로 성능을 비교하면서
어울리는 가전을 구매하고 싶은 고객

"저 자리에는 에어컨이 어울릴까?"
"삼성과 LG 에어컨 중 어떻게 기능이 좋을까?"

#성능 #어울림 #비교

서비스 부족을 느끼는 가전 구매 고객층은?



Actor3_Action2

온오프라인상 정보 비대칭 해소를 위해 손품, 발품 판다

Actor1_Action1

이전 구매 제품에 대해 불만 있어 새 제품 구매를 망설인다

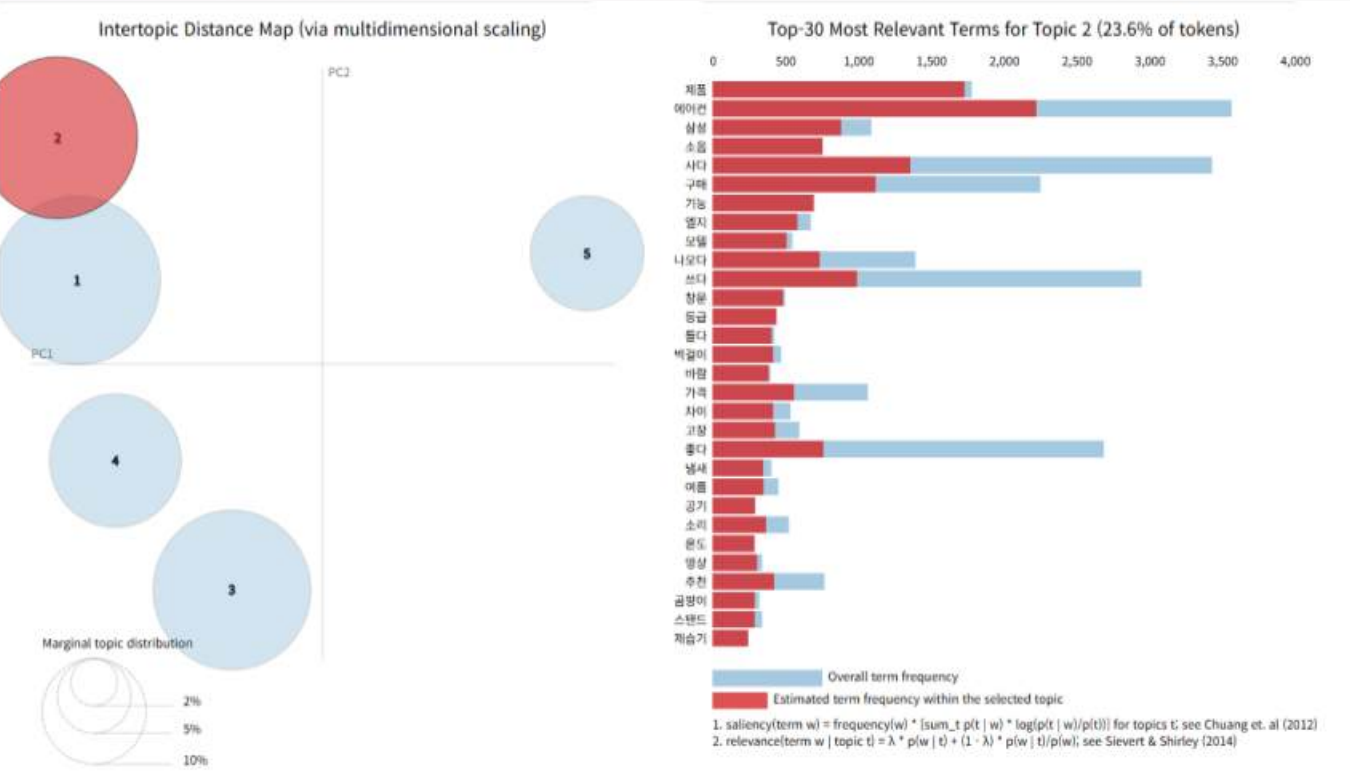
Actor0_Action2

실사용 리뷰를 기반으로 제품을 공부

이사를 앞두고 가전 설치 업체를 탐색

Persona Actor 0 action 2

실사용 리뷰를 기반으로 구매할 제품을 꼼꼼하게 공부하는 병관 씨



Keywords

- #사용자 리뷰
- #제품 정보
- #기능
- #모델
- #차이점
- #고장
- #비교



Content

- 저도 3월 정도에 이사를 계획하면서 기존에 있던 가전기기를 전부 버리고 전부 새걸로 교체하려고 하는데 한쪽 제조회사로 몰아서 살때 저렴하다고 많이 들었습니다. 잇섭님의 추가적인 어드바이스를 듣고나니 조만간 발품 팔아서 저도 준비해야 할 듯 싶습니다.
- 창문형 에어컨을 사려고 합니다 나무 창틀이라 삼성과 엘지는 포기했습니다 파세코가 좋아보이긴 한데 삼성보다 비싸네요 한 일은 저렴하긴 하나 자동 건조 시간이 짧고 소음이 크게 걸리네요 한일 제습기를 사용중인데 소음이 괴랄해서 신일 에어컨도 봤었는데 이걸 리뷰에 없네요. 어떤 걸 사야 할 지 고민됩니다. 혹시 추천하시는 제품 있을까요?
- 덕분에 엘지 에어컨 구입하는데 가장 큰 도움이 되었습니다 어제 설치했는데 특히 소음면에서 매우 만족스럽습니다. 추천 감사합니다.

Needs

- 제품 정보 비교 원함
- 실제 사용자 리뷰 필요로 함
- 소음 등 실사용 기준의 구체적 성능 비교 원함

Painpoints

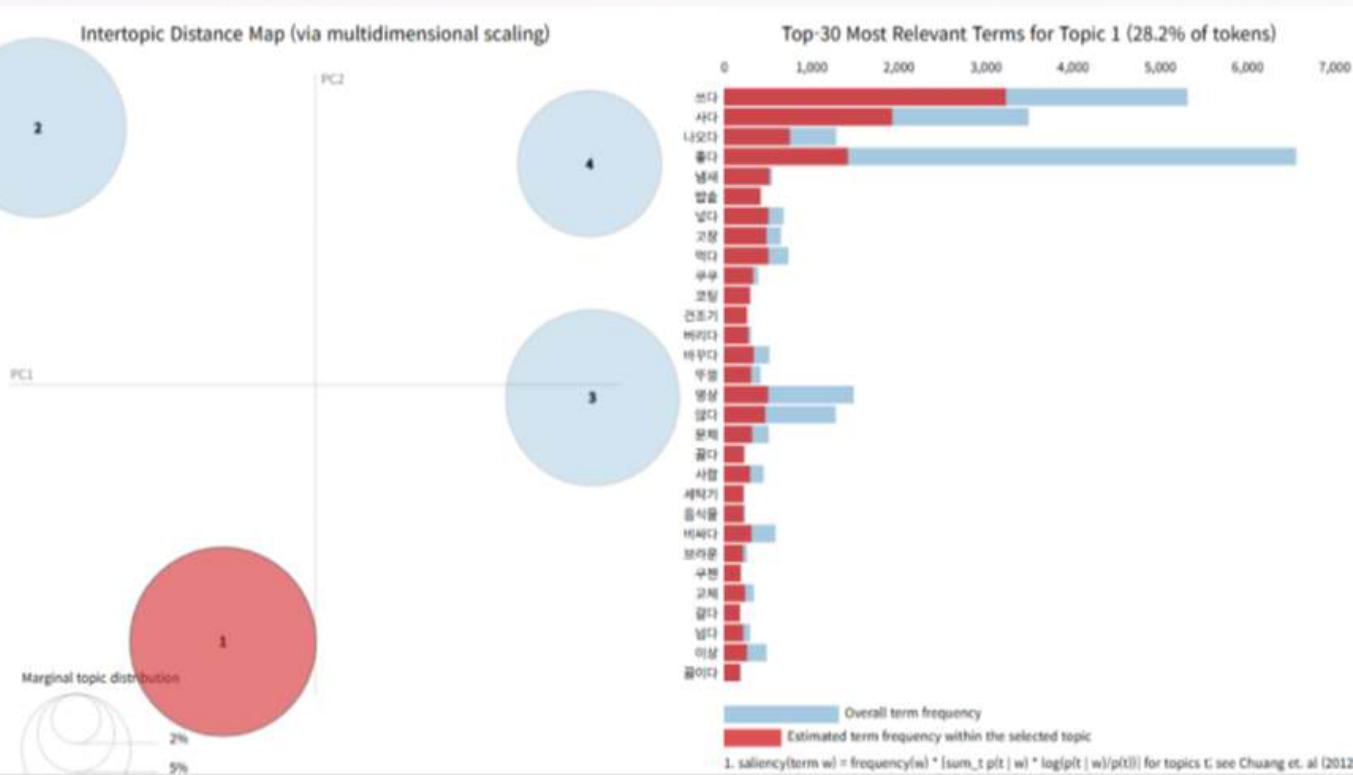
- 제품 종류가 너무 많음
- 스펙 차이를 이해하기 어려움
- 광고성 리뷰로 신뢰도 판별 어려움

Artifacts

- 유튜브 리뷰영상
- 온라인 매장 실사용 리뷰
- 오프라인 매장 실사용 리뷰

Persona Actor 1 action 1

불만족스러운 구매 경험으로 공식 정보보다 실사용 후기를 신뢰하는 지우 씨



Keywords

- #고장
- #교체
- #이상
- #냄새
- #바꾸다
- #버리다
- #문제



Content

- ‘영상잘봤습니다 유익한 영상 감사합니 다 혹시 S사 제품에서 식세기내부 벽면 에 검은색이물 묻어나오는지도 알수있 을까요 제품후기중에 물에적신티휴지로 닦았을때 검은색이물이 묻어나온다는 후기가있어서 궁금하네요’
- ‘H사 제품은 구매 안하는게 좋을듯 합니 다 전에 물걸레를 사용한 적이 있는데 a s받으려고 전화를 30 70통 했는데 안받 더군요 짜증 나서 폐기한 이후론 H사 제 품은 쳐다도 안봅니다’
- ‘동영상에서 알려주신대로 그대로 따라 했는데 에어프라이어 전선이 타더니 작 동을 안합니다 다른 분들이 혹시 보고 따 라해서 저같은 경우가 생기시지 않았으 면 합니다 지금 에어후라이어 사용된 지 1년도 안되었는데 버렸습니다’

Needs

- 다음 구매에서 같은 실수를 반복하지 않을 기준이 필요함
- 내가 겪었던 문제가 나만의 일인지 제품 자체의 문제인지 확인 필요함
- 브랜드를 믿지 않아 실사용자의 리뷰를 원함

Painpoints

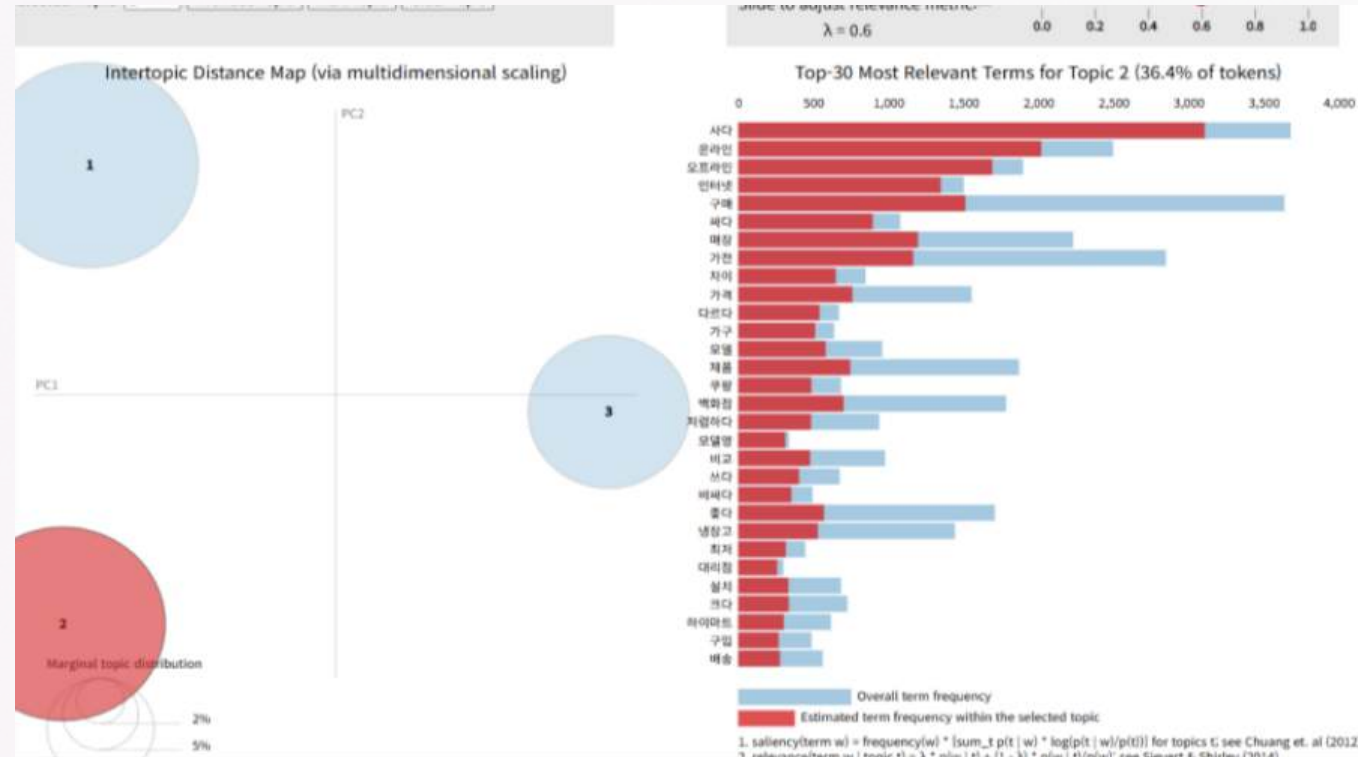
- 기대치와 실제 성능 간 괴리
- 제품 결함, 불만족스러운 AS에 의한 브랜드 불신

Artifacts

- 유튜브 구독
- 이마트
- 동영상

Persona Actor 3 action 2

온오프라인상 정보 비대칭 해소를 위해 손품, 발품을 파는 다빈 씨



Keywords

- #온라인
- #오프라인
- #최저가
- #모델명
- #차이
- #비교
- #다르다



Content

- ‘백화점용은 부품 고가로 들어갑니다. S사 백화점 납품용 300대 부품 리스트와 홈쇼핑용 500대 부품리스트가 **다릅니다.**’
- ‘엘지김치냉장고를 샀는데 옆면이 울통불통 하던데 백화점용과 대리점용 **쿠팡** 이랑 **다른가요?**’
- ‘**모델명이 같으면** 같은 제품이라는 게 맞지만 쿠팡 등 온라인전용제품 vs 백화점 및 로드샵 제품 **모델명이 달라요.** 온라인 제품 오프라인 제품이 무조건 같다고 오해에 소지가있습니다.’
- ‘**온라인에서 비싼 가전 단품을 구매할 때는** 후기 프로모션으로 라인업 등급별로 1~40만원까지 상품권 주는 경우가 있기에 **최저가 뿐만 아니라 판매자 게시글까지** 확인해봐야 합니다. **최저가라고** 덜렁 샀다가 **같은걸 샀는데** 검색해보니 저 사람은 뭘 받았다는데 **나는 없네?** 같은 경우도 생기거든요’

Needs

- **최적의 구매 타이밍, 장소 추천**
- **채널별 모델 차이 검증**
- **실질적 체감가 계산**

Painpoints

- **정보 탐색 피로**
- **온오프라인 모델명 불일치에 따른 불안**
- **오프라인 매장 발품의 피로**

Artifacts

- 하이마트
- 오프라인 매장
- 온라인 매장
- 백화점
- 대리점



오프라인 구매 과정에서 느끼는 핵심 만족 요소를 온라인 서비스로 구현해 D2C 채널 강화

Actor 4_ Action 1

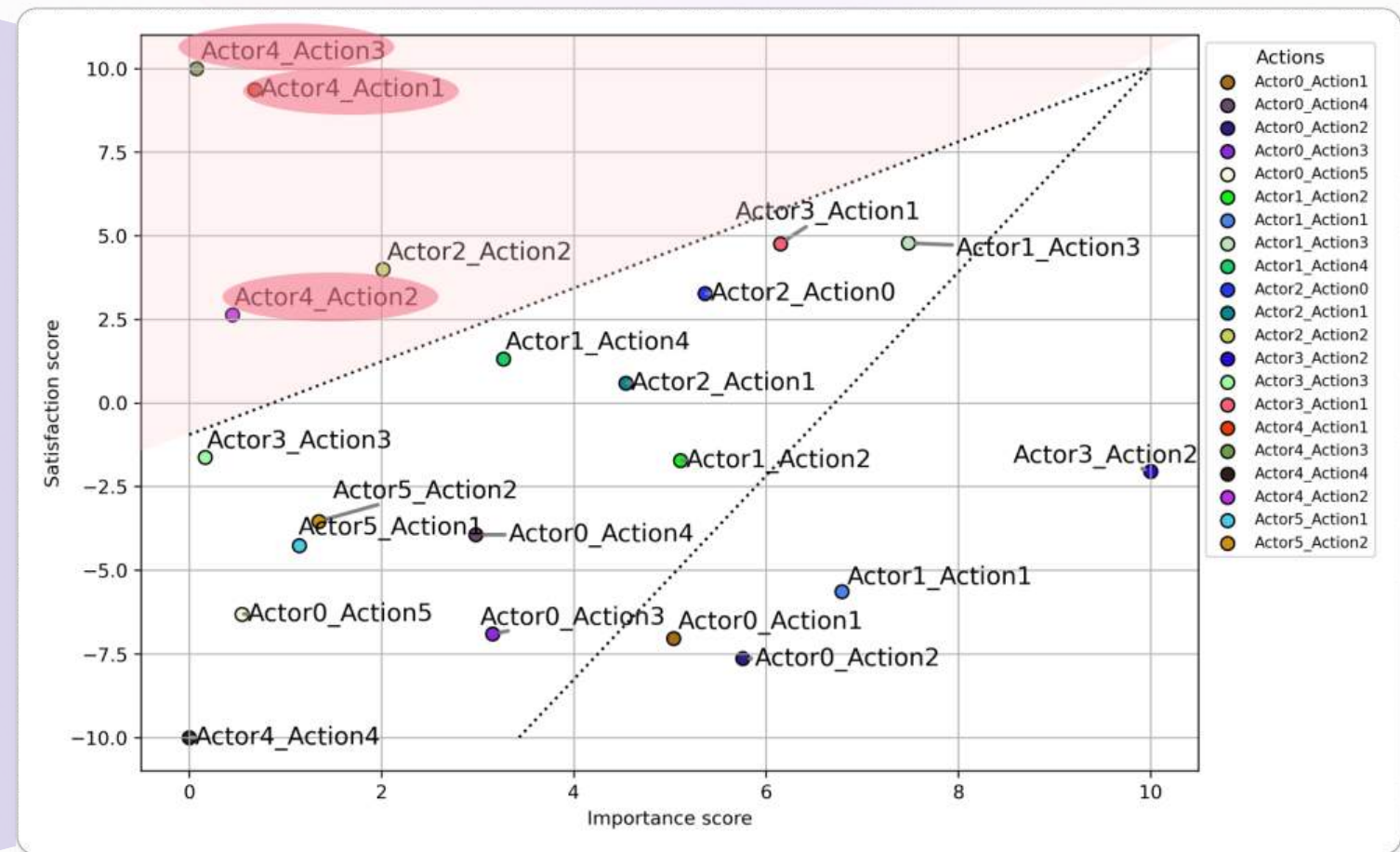
담당자의 상세한 설명(혜택, 할인, 기능 등)에 만족하는 고객

Actor 4_ Action 2

다품목 구매 예정으로 오프라인에서 혜택 및 할인을 많이 받고 싶어하는 고객

Actor 4_ Action 3

이사 등 특별한 사유로 다품목을 위해 견적을 원하는 고객



오프라인 구매 과정에서 느끼는 핵심 만족 요소를 온라인 서비스로 구현해 D2C 채널 강화

Actor 4_ Action 3

담당자의 상세한 설명(혜택, 할인, 기능 등)에 만족하는 고객

Data Insight

Actor 4_ Action 1

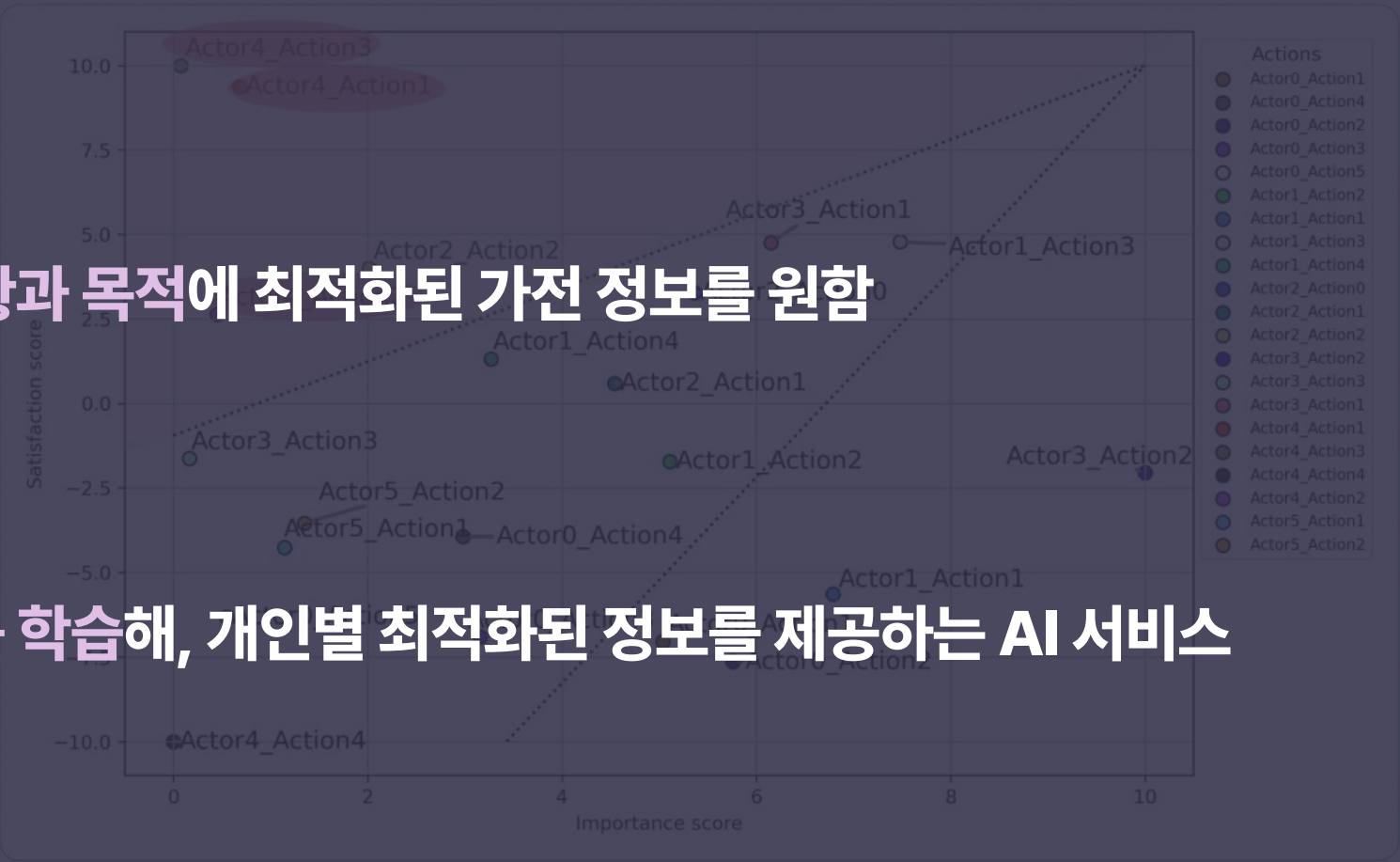
이사 등 특별한 사유로 다품목을 위해 견적을 원하는 고객

Service Concept

다품목 구매 예정으로 오프라인에서 혜택 및 할인을 많이 받고 싶어하는 고객

고객은 자신의 구매 상황과 목적에 최적화된 가전 정보를 원함

고객의 구매 페르소나를 학습해, 개인별 최적화된 정보를 제공하는 AI 서비스



L-PICK

당신을 위한 가장 똑똑한 AI 이커머스 솔루션





가전 구매 시 어떤 기준을 중요하게 생각하시나요?

가격 / 성능 / 디자인의 우선순위를 차례대로 선택해주세요

가격

성능

디자인

선택지에 없다면 중요하게 생각하는 구매 기준을 입력해 주세요.



지셀 상담사

현명한 선택을 위한 맞춤 가전 가이드



안녕하세요! 맞춤 가전 상담사 지셀입니다.
고객님에게 맞는 최적의 제품을 찾아드리겠습니다.
관심 있는 제품군을 편하게 클릭주세요 🍷

어떤 가전 제품이 필요하신가요?

관심 있는 카테고리들 모두 선택해주세요

중복 선택 가능



다음 질문

지셀 상담사

현명한 선택을 위한 맞춤 가전 가이드



고객님의 선택과 조건을 기반으로 가장 비용 효율
적인 구매 시점과 방법을 분석했어요.
지금부터 최적의 선택을 함께 확인해보실까요?

가장 합리적 으로 구매할 수 있는 방법이에요!

AI 추천 패키지 01 · 5개 제품 구성 기준으로 온라인/오프라인 구매 조건을
비교해 추천드렸어요.



가장 저렴한 온라인 구매 시점

3월 15일 lge.com에서 신한카드로 결제하시면 가장 저렴해
요!
3월 15일에 카카오톡 알림 보내드릴까요?



가장 유리한 오프라인 구매 법칙

4월 8일 LG 베스트샵 홍대입구 지점에서 사시면 가장 저렴
해요!
4월 8일 방문 예약 도와드릴까요?

가전을 구매할 때 원한 점을 입력해 주세요.



추가로 원하는 구매 방식이나 방문 희망 매장을 입력해주시면 더 맞춤형드릴 수 있어요.

👉 홈으로 돌아가기

초개인화 AI 이커머스 솔루션, L-Pick

AI가 고객 페르소나를 자동 분류해
초개인화된 가전 추천, 실사용자 리뷰 요약, 최적 구매 경로를 안내합니다.

L-Pick 은 번거로운 가전 탐색 과정을 줄여,
고객에게 차별화된 구매 경험을 선사합니다.



가전 구매 시 어떤 기준을 중요하게 생각하시나요?

가격 / 성능 / 디자인의 우선순위를 차례대로 선택해주세요



가격



성능



디자인

선택지에 없다면 중요하게 생각하는 구매 기준을 입력해 주세요.

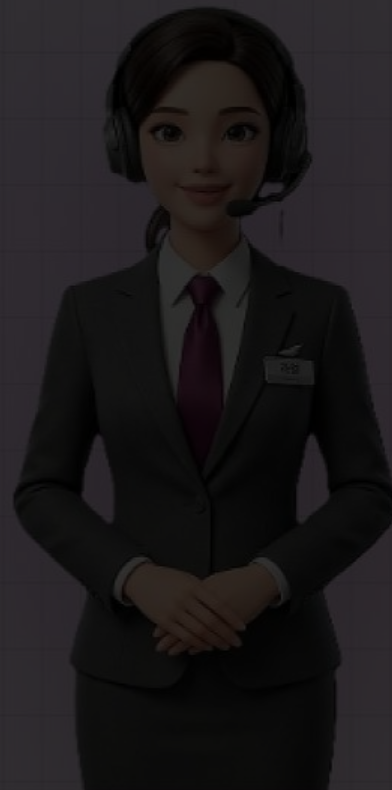


사용자 페르소나 학습

질문을 통해 **고객 가전 구매** 성향 정보 수집

오프라인 고객 만족 요소인 **초개인화된 정보** 제공 구현

Actor 4_Action 1,2,3



가전 구매 시
어떤 기준을 중요하게 생각하시나요?

가격 / 성능 / 디자인의 우선순위를 차례대로 선택해주세요



가격



성능



디자인

선택지에 없다면 중요하게 생각하는 구매 기준을 입력해 주세요.



지셀 상담사

현명한 신혼 부부를 위한 맞춤 가전 가이드

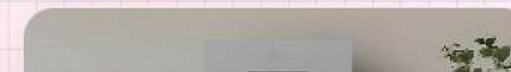


안녕하세요! 맞춤 가전 상담사 지셀입니다.
고객님에게 맞는 최적의 제품을 찾아드리겠습니다.
관심 있는 제품군을 편하게 골라주세요 😊

어떤 가전 제품이 필요하신가요?

관심 있는 카테고리를 모두 선택해주세요

중복 선택 가능



다음 질문

페르소나 맞춤 셀러

분류된 페르소나별로
초개인화된 서비스 제공

Ex. 페르소나 분류
알뜰한 1인 가구
맞벌이 부부
활동이 많은 대가족
Actor 4_Action 1,2,3

지셀 상담사

현명한 신혼 부부를 위한 맞춤 가전 가이드



안녕하세요! 맞춤 가전 상담사 지셀입니다.
고객님에게 맞는 최적의 제품을 찾아드리겠습니다.
관심 있는 제품군을 편하게 골라주세요 😊

어떤 가전 제품이 필요하신가요?

관심 있는 카테고리를 모두 선택해주세요

중복 선택 가능



다음 질문

고객님을 위한 맞춤 패키지

답변 내용을 바탕으로 선별한 최적의 제품 조합입니다

설정하신 예산 내에서 라이프스타일에 맞춘
최적의 가전 패키지를 추천해 드릴게요.

🔄 다시 추천

AI 추천 패키지 01



LG 디오스 오브제컬렉션 냉장고 Fit & Max

690L / 1등급

AI **실사용자 리뷰** 키워드

인테리어

키친핏

수납활용



LG 올레드 evo (스탠드형)

C4 / 138cm

AI **실사용자 리뷰** 키워드

화질

음질

크기

성능



LG 휘센 오브제컬렉션 사계절에어컨 2in1 (타워 히트)

스탠드형 / 65.9㎡

AI **실사용자 리뷰** 키워드

전력효율

냉방_난방

기능

소음



LG 디오스 오브제컬렉션 전자레인지

25L

AI **실사용자 리뷰** 키워드

디자인

조리속도

사용편

AI 추천 패키지 02



LG 퓨리케어 오브제컬렉션 하이드로타워



LG 휘센 오브제컬렉션 사계절에어컨 2in1 (타워 히트)



LG 디오스 오브제컬렉션 냉장고 Fit & Max



LG 올레드 evo (스탠드형)

☆ AI 맞춤 추천 완료

고객님을 위한 맞춤 패키지 AI 자동 가전 추천

답변 내용을 바탕으로 선별한 최적의 제품 조합입니다

설정하신 예산 내에서 라이프스타일에 맞춤
최적의 가전 패키지를 추천해 드릴게요.

실사용자 리뷰를 분석해 요약

진실성 있는 리뷰를 찾으려는 니즈 반영

Actor 0_Action 2, Actor 1_Action 1

다시 추천

AI 추천
패키지 01



LG 디오스 오브제컬렉션 냉장고 Fit & Max

690L / 1등급

AI 실사용자 리뷰 키워드

인테리어

키친핏

수납활용

LG 올레드

C4 / 138cm

AI 실사용자 리뷰

화질



리뷰 키워드 상세

인테리어

LG 디오스 오브제컬렉션 냉장고 Fit & Max

실사용자 리뷰

들어가는 Fit 설계 덕분에 빌트인처럼 깔끔하게 자리 잡아 주방이 훨씬 넓고 정돈된 느낌이예요. 특히 에센스 화이트 컬러는 과하지 않으면서도 고급스럽고, 어떤 주방 인테리어에도 자연스럽게 어울려요. 군더더기 없는 미니멀한 디자인이라 가전이 튀지 않고, 마치 원래부터 주방의 일부였던 것처럼 조화롭게 녹아듭니다. 디자인만 놓고 봐도 정말 만족도가 높아요.

디자인도 이쁘고 1등급으로 환급도 받아 만족스럽습니다. 다만 매직스페이스가 없어서 조금 아쉽습니다.

일단 예뻐요. 인테리어하고 이사하면서 산건데 너무 맘에들어요. 냉장고 스펙이 여러가지 있어서 혹시라도 가격이 제일 싸서 품질이 안 좋을까 조금 걱정했지만 기본이 중요하고 한달 이상 썼는데 전혀 문제없어요. 김치냉장고와 세트로 냉장 고장도 정확한 치수로 들어놓으니너무 만족스럽습니다.

관련 유튜브 리뷰



유튜브 리뷰 보러가기 1



유튜브 리뷰 보러가기 2



LG 디오스 오브제컬렉션 사계절에어컨 2in1 (타

난방

기능

소음



LG 디오스 오브제컬렉션 전자레

25L

AI 실사용자 리뷰 키워드

디자인

조리속도

사용편

AI 추천
패키지 02



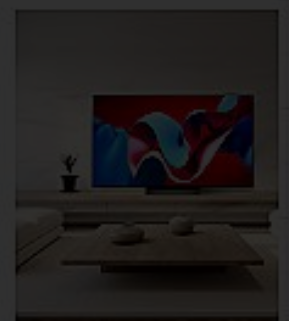
LG 퓨리케어 오브제컬렉션 하이드루타워



LG 올레드 오브제컬렉션 사계절에어컨 2in1 (타



LG 디오스 오브제컬렉션 냉장고 Fit & Max



LG 올레드 evo (스탠드형)

지셀 상담사

현명한 신혼 부부를 위한 맞춤 가전 가이드



고객님의 선택과 조건을 기반으로 가장 비용 효율적인 구매 시점과 방법을 분석했어요.
지금부터 최적의 선택을 함께 확인해보실까요?

가장 합리적 으로 구매할 수 있는 방법이에요!

AI 추천 패키지 01 · 5개 제품 구성 기준으로 온라인/오프라인 구매 조건을
비교해 추천드렸어요.



가장 저렴한 온라인 구매 시점

3월 15일 lge.com에서 신한카드로 결제하시면 가장 저렴해요!

3월 15일에 카카오톡 알림 보내드릴까요?



가장 유리한 오프라인 구매 방식

4월 8일 LG 베스트샵 홍대입구 지점에서 사시면 가장 저렴해요!

4월 8일 방문 예약 도와드릴까요?

가전을 구매할 때 힘든 점을 입력해 주세요.

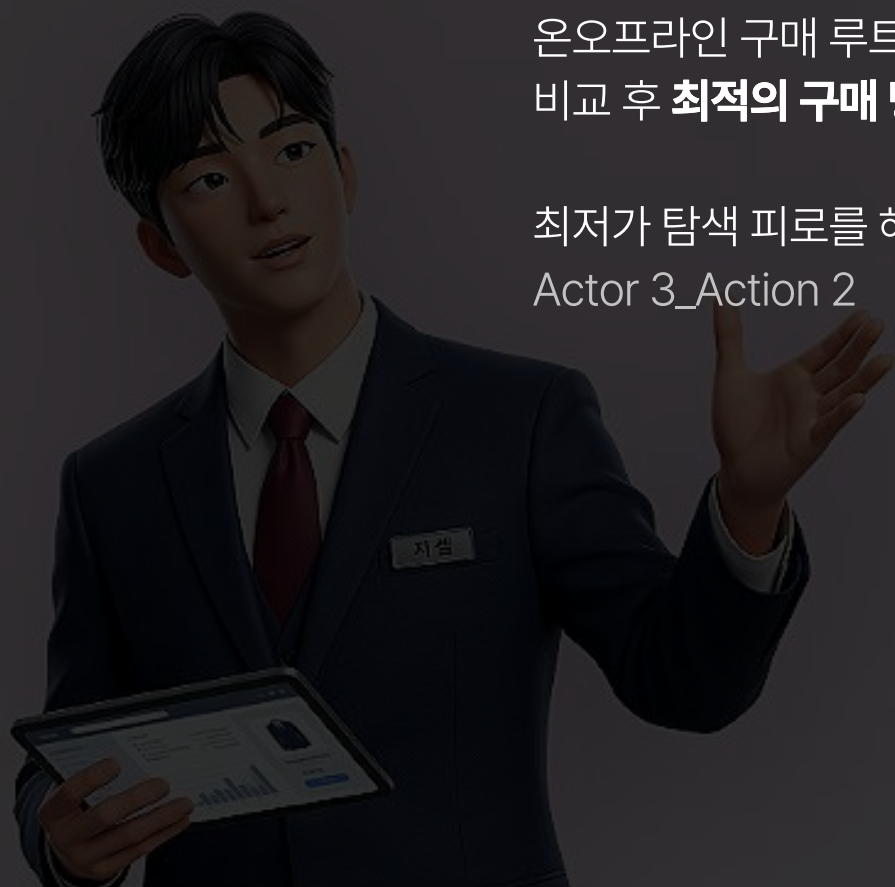


추가로 원하는 구매 방식이나 방문 희망 매장을 입력해주시면 더 맞춰드릴 수 있어요.

🏠 [홈으로 돌아가기](#)

지셀 상담사

현명한 신혼 부부를 위한 맞춤 가전 가이드



최적 구매 경로 안내

온오프라인 구매 루트 및 가격
비교 후 **최적의 구매 방법** 안내

최저가 탐색 피로를 해소
Actor 3_Action 2

고객님의 선택과 조건을 기반으로 가장 비용 효율
적인 구매 시점과 방법을 분석했어요.
지금부터 최적의 선택을 함께 확인해보실까요?

가장 합리적 으로 구매할 수 있는 방법이에요!

AI 추천 패키지 01 · 5개 제품 구성 기준으로 온라인/오프라인 구매 조건을
비교해 추천드렸어요.



가장 저렴한 온라인 구매 시점

3월 15일 lge.com에서 신한카드로 결제하시면 가장 저렴해
요!
3월 15일에 카카오톡 알림 보내드릴까요?



가장 유리한 오프라인 구매 방식

4월 8일 LG 베스트샵 홍대입구 지점에서 사시면 가장 저렴
해요!
4월 8일 방문 예약 도와드릴까요?

가전을 구매할 때 힘든 점을 입력해 주세요.



추가로 원하는 구매 방식이나 방문 희망 매장을 입력해주시면 더 맞춰드릴 수 있어요.

🏠 **홈으로 돌아가기**

통합적 고객 행동 분석

더 깊은 사용자 경험 (정신적)

- 나를 이해하는 AI 셀러의 맞춤 추천
- 예산, 생활패턴 기반 개인화 경험
- 온라인에서 느끼는 오프라인 감성
- 리뷰 중 사용자에게 필요한 정보만 선별하여 탐색의 피로 해소
- 사용자 맞춤 제품 선택 기준 안내

더 선명한 사용자 경험 (물리적)

- AI 서비스를 눈에 보이는 실체로 만들어주는 셀러 캐릭터
 - 단축시간 및 노력에 대한 시각화
ex. 고객님의 근방 베스트샵 n개 검색 결과
- 기능중심 설명(dB, W 등)을 보완하는 실제 사용 리뷰 정보화
 - 선택지 제시
 - 프로그레스 바(이탈방지)

4D-CX

더 넓은 사용자 경험 (문화적)

- 동일 제품 사용자의 리뷰 수집 및 요약
- 생활패턴이 유사한 사용자가 사용한 제품 추천
- 구매자들의 다품목 구매 성향을 고려한 패키지 추천
- 페르소나별 가전 구매 기준 표시

더 큰 사용자 경험 (시스템)

- LGE 닷컴과 베스트샵 연동을 통한 실시간 구매 최적화 시스템
 - LGE 닷컴 리뷰데이터 요약 및 제공
 - 카드 혜택 연결을 통한 제휴 할인 안내

F.U.N. 요소

[정보 가독성 및 시인성이 떨어지는 현재 AI 챗봇]

혼수 냉각과 추위에 주제요	1인 가구 에어컨 추천에 주제요
오후 4:57	오후 4:58
말씀해주신 결혼 준비용 냉방고와 잘 어울리는 프리미엄 라인에서 몇 가지 제품 을 추천드릴 수 있어요.	고객님께서 찾으시는 1인 가구 전용 모델과 완전히 동일한 조건은 아니지만, 1 인 가구에 잘 어울리는 가까운 사양의 에어컨들을 골라 안내드릴게요.
한눈에 살펴보기	한눈에 살펴보기
• LG SIGNATURE 얼음정수 냉방고 (J842ND79) 840L 대용량과 직수 얼음-정수 시스템, 솔리드 스테인리스 디자인으로 신 선 주방용 한 번에 고급스럽게 완성하고 싶은 고객님께 잘 맞아요. 매직스메 이스와 얼어두수크너, 대용량 바스켓 덕분에 찜을 넉넉히 봐도 식재료 정리 가 아주 편해요.	• LG 휘센 벽걸이에어컨 (SQ06EA1WCS) 약 18.7㎡(안방·원룸 크기) 냉방이 가능한 1인 가구 침실·원룸에 달맞고, 아이소파라워·저소음·AI 비바람·강력 제습으로 작은 공간도 빠르게 시원하고 조용하게 써보고 싶은 고객님께 잘 맞아요. 자동 열교환기 세척과 AI 건조 기능이 있어, 혼자 살아도 에어컨 관리 부담을 줄이고 싶은 분께 특히 좋아 요.
• LG SIGNATURE 상냉장고 (F904ND79E) 905L 초대용량에 상냉장/하냉동 4도어 구조와 신선 초퍼터 나중애 가족이 늘어나도 여유 있게 쓰기 좋아요. 1등급 수준애 높은 에너지 효율과 오토 스 마트 도어 기능으로 전기요금과 사용 편의성을 함께 챙기고 싶은 고객님께 추천해요.	• LG 휘센 오브제컬렉션 세 에어컨 2in1 (1시리즈) (FQ17F1C1ED2) 거실+방 2곳을 함께 쓰는 집이나, 앞으로 이사·확장을 고려하시는 1인 가구 라면 스탠드+벽걸이 2in1 구성이 경제적이예요. 약 56.9㎡ 냉방, 반려들 물을 위한 벡케어 모드, 자동 열교환기 세척으로 넓은 집에서도 시원함·위생 ·인테리어를 한 번에 챙기기 좋아요.
• LG 디오스 오브제컬렉션 STEM 베이직 냉방고 Fit & Max (노크온) & LG 디오스 오브제컬렉션 김치독자 Fit & Max (R1081SVV357) 냉방고와 김치냉장고 둘 한 번에 맞춰 인테리어까지 통일하고 싶은 신혼 부 부께 잘 어울려요. Fit & Max 타입이라 가구처럼 일차 설치가 가능해 built-in 느낌을 내고, STEM 시스템으로 얼음·냉가·위생까지 신경 쓰는 고 급님께 적합해요.	• LG 휘센 AI 오브제컬렉션 뷰비 에어컨 2in1 (1시리즈) (FQ17FU1ED2) 솔직한 디자인과 교체 가능한 루 커메어로 인테리어를 중시하는 1인 가구에 게 어울리고, 마한가지로 56.9㎡ 냉방이 가능해 넓은 침·거실 위주 생활 에도 여유 있어요. AI 바람·벡케어·강력 제습과 간편 플린터 구조로, 오래 써 도 깔끔하게 관리하고 싶은 고객님께 추천해요.

[온라인 가전 구매 과정을 가이드하는 L-Pick]

The collage displays four mobile app screens for a home appliance company:

- Top Left:** A female customer service representative in a headset.
- Top Right:** A screen with a 27% progress bar and the text: "가전 구매 시 어떤 기준을 중요하게 생각하시나요?" (When buying home appliances, what standards do you think are important?). Below it, a note says: "가격 / 성능 / 디자인의 두 기준치를 차례대로 선택해주세요" (Please select the two criteria of price / performance / design in order).
- Bottom Left:** A screen titled "지셀 상담사" (Zicell Sales Representative) featuring a male representative. Text below him says: "원만한 상담 부탁을 위한 질문 기원합니다." (We wish you good questions for a smooth consultation). At the bottom, it says: "만능주세로 모든 가전 상담사 지셀입니다. 고객님께 맞는 최상의 제품을 찾아드리겠습니다. 원하시는 제품을 한여석에 골라주세요 🍷" (I'm Zicell, the all-knowing sales representative for all home appliances. We'll find the best product for you. Please choose the product you want on the one-handed seat 🍷).
- Bottom Right:** A screen titled "어떤 가전 제품이 필요하신가요?" (Which home appliance do you need?). It lists categories: "냉장고는 가전2018를 골라 선택해주세요" (Please select a refrigerator from gajen2018). Below is a grid of product images: TV, Air Purifier, Refrigerator, Washing Machine, Dryer, and others. A pink button at the bottom says "가전 선택" (Select Home Appliance).

FIRST

UNIQUE

NEW

캐릭터 UI를 통한 AI와의 실감나는 상호작용

버튼형 UI 도입을 통해 대화형 AI의 자유 입력 형식에서 오는 사용자 불편 해소

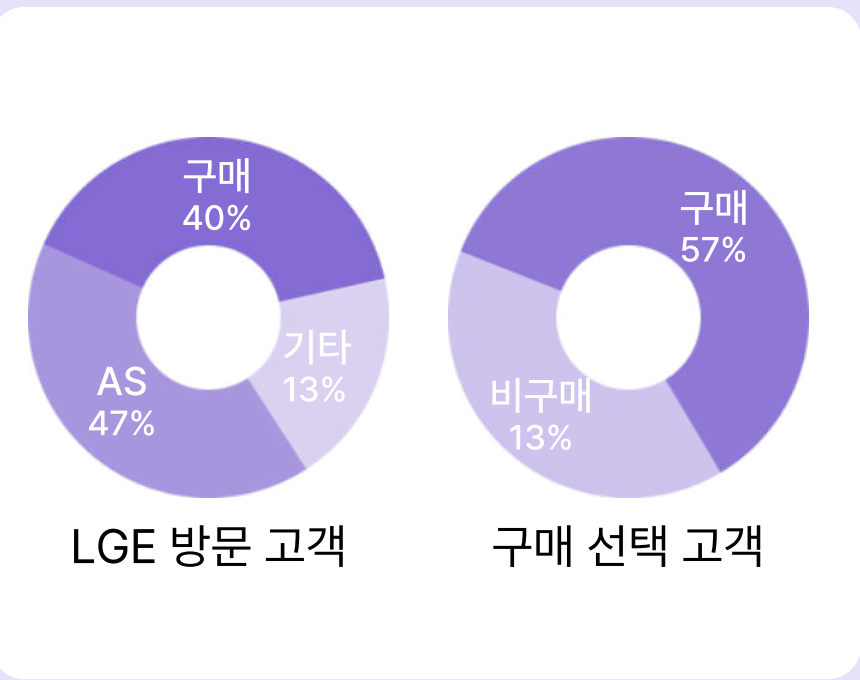
오프라인 매장의 다채로운 구매 경험을 온라인 자사몰에서 재현

목표 달성 확인

01

L-Pick 이용 고객 중 실제 구매로 이어진 비율

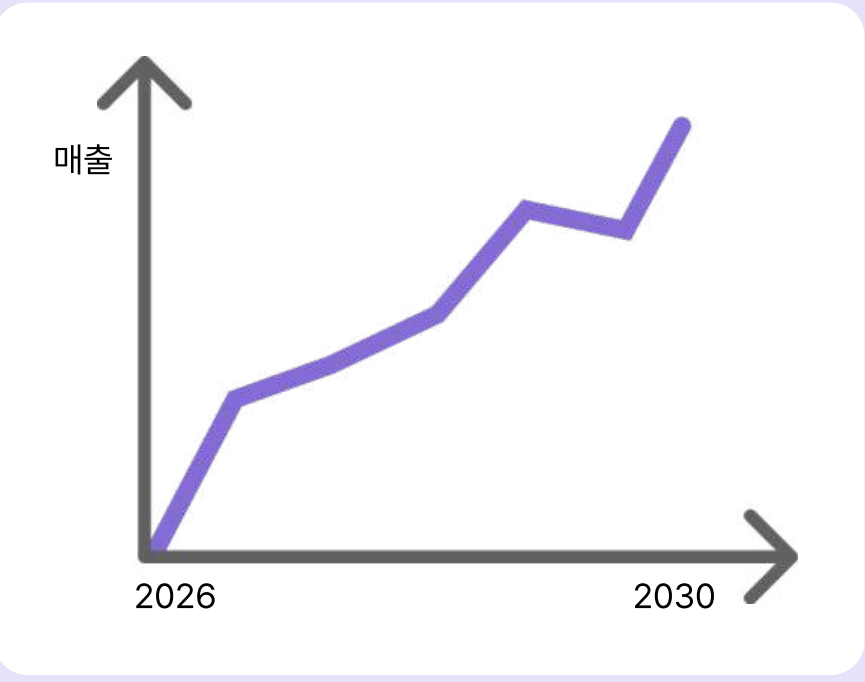
L-Pick 사용 후 실제 구매로 이어지는 비율



02

D2C 활성화 수준

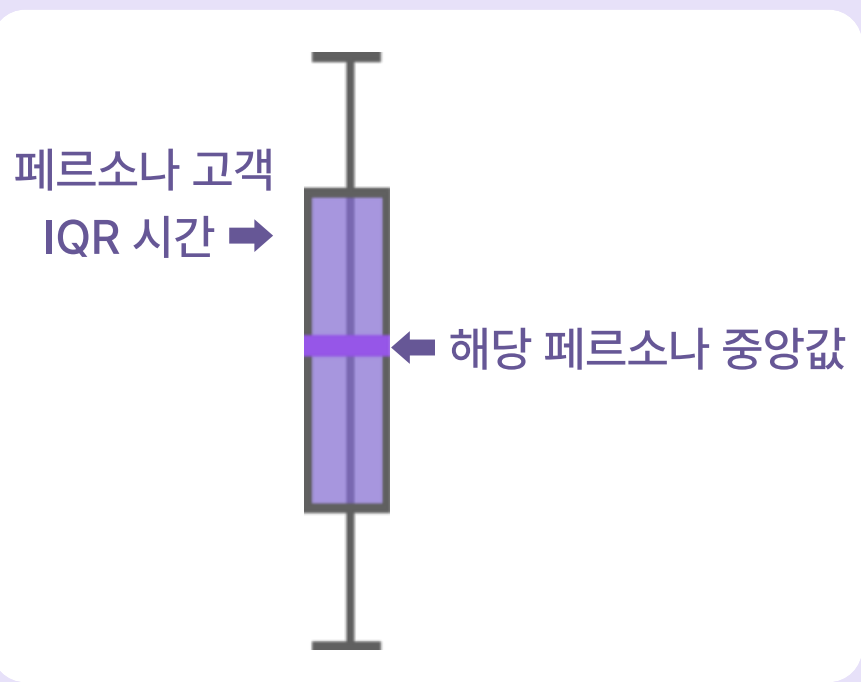
D2C 판매 성과 지표



03

실제 고객과 L-Pick이 파악한 페르소나의 매칭률

셀러에게 추천받은 제품 사용률(ThinQ 연동)

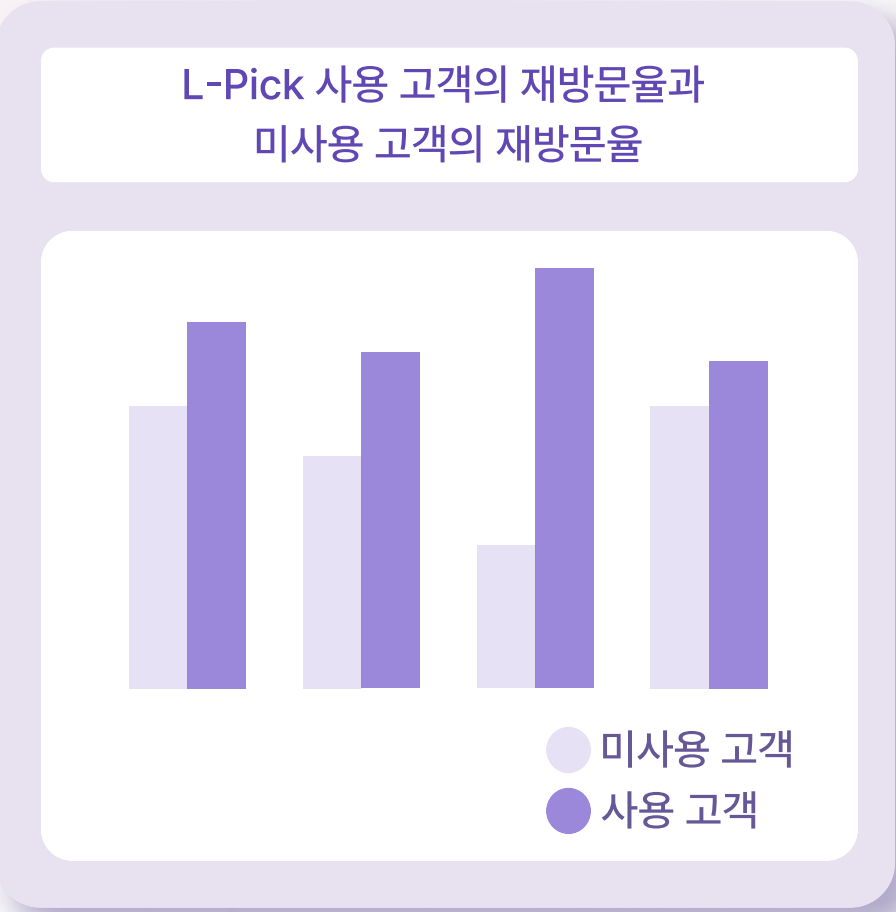


목표 달성 확인

04 L-Pick 통해 구매한 고객의 만족 지표



05 L-Pick 사용 고객의 재방문율과 미사용 고객의 재방문율 비교 지표



감사합니다.



| Appendix | 코드

yter 0311_유투브_가전_온라인_오프라인 Last Checkpoint: 7 days ago

```
View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
Code

subtitles = []

for n, i in enumerate(tuple(youtube_link_list)):
    mini_sub = ""
    driver.get(i)
    wait = WebDriverWait(driver, 15)

    try:
        target_xpath = '//*[@id="movie_player"]/div[1]/video'
        wait.until(EC.element_to_be_clickable((By.XPATH, target_xpath))).click()

        expand_xpath = '//*[@id="expand"]'
        wait.until(EC.element_to_be_clickable((By.XPATH, expand_xpath))).click()

        script_xpath = '//*[@id="primary-button"]/yt-button-renderer'
        wait.until(EC.element_to_be_clickable((By.XPATH, script_xpath))).click()

    except Exception as e:
        print(f"({n})현재 영상 버튼 조작 중 오류 발생")
        subtitles.append(None)
        continue

    try:
        selectors = [
            'elements-container > yt-d-transcript-segment-renderer > div > yt-formatted-string',
            'Accountis > macro-markers-panel-item-view-model > timeline-item-view-model > div > transcript-segment-view-model > span'
        ]

        found = False
        for selector in selectors:
            try:
                'elements-container > yt-d-transcript-segment-renderer > div > yt-formatted-string',
                'Accountis > macro-markers-panel-item-view-model > timeline-item-view-model > div > transcript-segment-view-model > span'
            ]

            found = False
            for selector in selectors:
                try:
                    wait.until(EC.presence_of_all_elements_located((By.CSS_SELECTOR, selector)))
                    soup = bs(driver.page_source, "html.parser")
                    elements = soup.select(selector)

                
```

```
from bs4 import BeautifulSoup

def get_store_list():
    driver = build_driver()
    stores_data = []

    try:
        driver.get(BASE_URL)
        time.sleep(5)

        html = driver.page_source
        soup = BeautifulSoup(html, "html.parser")

        store_links = soup.select("a.base-line[data-orgcode]")
        print("오입란 횡랑 침수 수:", len(store_links))

        for a in store_links:
            org_code = (a.get("data-orgcode") or "").strip()

            strong = a.select_one("strong")
            title = clean_text(strong.get_text(strip=True) if strong else "")

            if title and org_code:
                stores_data.append({
                    "title": title,
                    "org_code": org_code
                })

    finally:
        driver.quit()

# 결과 출력
seen = set()
uniq = []

for row in stores_data:
    key = (row["title"], row["org_code"])
    if key not in seen:
        seen.add(key)
        uniq.append(row)

print("윗론 환란 횡랑 수:", len(uniq))
return uniq

stores = get_store_list()
print("len(stores) =", len(stores))
print(stores[:5])
```

데이터 전처리하기

```
import re

def clean_text(text):
    # URL 제거
    text = re.sub("http://", "", text)

    # HTML 태그 제거
    text = re.sub("<.*>", "", text)

    # 공백 제거
    text = re.sub(" ", "", text)

    # 특수문자 제거
    text = re.sub("[^a-zA-Z0-9]", "", text)

    # 여러 줄 한 줄로
    text = re.sub("\n", "\n", text)

    # 마침표 제거
    text = re.sub(" ", "", text)

    return text.strip()
```

모델 불러오기

```
import torch
import warnings
from sentence_transformers import SentenceTransformer

warnings.filterwarnings("ignore")

# GPU 사용 가능 여부 확인
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using device: {device}")

model = SentenceTransformer("intfloat/multilingual-es-large-instruct", device=device)

# Content 필터링 인스턴스 생성
texts = df['content'].astype(str).tolist()

# 임베딩 생성
embeddings = model.encode(texts, normalize_embeddings=True)

# DataFrame에 vector 컬럼 추가
df['vector'] = embeddings.tolist()

# 출력
df.head()
```

t-SNE

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import os

# t-SNE 모델 설정
n_components = 2  # 시각화를 위한 2차원 공간으로 축소
perplexity, learning_rate, n_iter_max = 30, 200, 1000  # perplexity, learning_rate, n_iter_max은 TSNE의 하이퍼파라미터
random_state = 42  # 랜덤 시드 설정
link_model = TSNE(n_components=2, random_state=42, perplexity=30, learning_rate=200, n_iter_max=1000)

# Doc2Vec 결과에 t-SNE 적용
df['vector'] = link_model.fit_transform(np.array(list(df['vector'])))

# 결과 출력
df['time_s'] = time_embedding[:, 0]
df['time_y'] = time_embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(10, 10))
sns.scatterplot(
    x="time_s",
    y="time_y",
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title("t-SNE Projection of ES Embeddings", fontsize=16)
plt.xlabel("t-SNE Dimension 1", fontsize=12)
plt.ylabel("t-SNE Dimension 2", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
time_filename = os.path.join(output_dir, 'time_projection.png')
plt.savefig(time_filename)
plt.show()
print(f"t-SNE plot saved to {time_filename}")
plt.close() # Close the plot to free memory
```

UMAP

```
import umap
import matplotlib.pyplot as plt
import seaborn as sns

umap_reducer = umap.UMAP(n_neighbors=15, min_dist=0.5)

embedding = umap_reducer.fit_transform(np.array(list(df['vector'])))

result_df['umap_s'] = embedding[:, 0]
result_df['umap_y'] = embedding[:, 1]

plt.figure(figsize=(10, 10))
sns.scatterplot(
    x="umap_s",
    y="umap_y",
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title("UMAP Projection of Doc2Vec Embeddings", fontsize=16)
plt.xlabel("UMAP Dimension 1", fontsize=12)
plt.ylabel("UMAP Dimension 2", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
umap_filename = os.path.join(output_dir, 'umap_projection.png')
plt.savefig(umap_filename)
plt.show()
print(f"UMAP plot saved to {umap_filename}")
plt.close() # Close the plot to free memory
```

t-SNE

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import os

# t-SNE 모델 설정
n_components = 2  # 시각화를 위한 2차원 공간으로 축소
perplexity, learning_rate, n_iter_max = 30, 200, 1000  # perplexity, learning_rate, n_iter_max은 TSNE의 하이퍼파라미터
random_state = 42  # 랜덤 시드 설정
link_model = TSNE(n_components=2, random_state=42, perplexity=30, learning_rate=200, n_iter_max=1000)

# Doc2Vec 결과에 t-SNE 적용
df['vector'] = link_model.fit_transform(np.array(list(df['vector'])))

# 결과 출력
df['time_s'] = time_embedding[:, 0]
df['time_y'] = time_embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(10, 10))
sns.scatterplot(
    x="time_s",
    y="time_y",
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title("t-SNE Projection of ES Embeddings", fontsize=16)
plt.xlabel("t-SNE Dimension 1", fontsize=12)
plt.ylabel("t-SNE Dimension 2", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
time_filename = os.path.join(output_dir, 'time_projection.png')
plt.savefig(time_filename)
plt.show()
print(f"t-SNE plot saved to {time_filename}")
plt.close() # Close the plot to free memory
```

UMAP으로 차원축소

```
import numpy as np
import umap
import matplotlib.pyplot as plt
import os

# vector 컬럼 = numpy 배열
# Convert string representations of lists to actual lists of floats
X = np.array(ast.literal_eval(x) for x in df['vector'])

# UMAP 모델 생성
reducer = umap.UMAP(
    n_neighbors=15, # 이웃 개수(기본값 10)
    min_dist=0.5, # 점 간 거리(기본값 0.5)
    n_components=2,
    metric='euclidean',
    random_state=42
)

X_umap = reducer.fit_transform(X)

# 결과를 Matplotlib로 시각화
df['umap_s'] = X_umap[:, 0]
df['umap_y'] = X_umap[:, 1]

# 시각화
plt.figure(figsize=(10, 7))
plt.scatter(df['umap_s'], df['umap_y'], s=40, alpha=0.7)
plt.xlabel("UMAP-1")
plt.ylabel("UMAP-2")
plt.title("UMAP Projection of text embeddings")
plt.show()
```

HDBSCAN으로 클러스터링

```
import hdbscan
import matplotlib.pyplot as plt

# HDBSCAN 클러스터링
clusterer = hdbscan.HDBSCAN(
    min_cluster_size=15,
    min_samples=5,
    metric='euclidean',
    cluster_selection_method='eom',
    prediction_data=True
)

labels = clusterer.fit_predict(df[['umap_s', 'umap_y']])

# 결과 출력
df['cluster'] = labels

# 클러스터의 개수 확인 (-1은 noise)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = len(labels == -1).sum()

print(f"클러스터 개수: {n_clusters}")
print(f"노이즈 개수: {n_noise}")
print(df['cluster'].value_counts().sort_index())
```

HDBSCAN 시각화

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 7))

for label in sorted(df['cluster'].unique()):
    subset = df[df['cluster'] == label]

    if label == -1:
        plt.scatter(
            subset['umap_s'], subset['umap_y'],
            s=12, alpha=0.5, c='lightgray', label="noise"
        )
    else:
        plt.scatter(
            subset['umap_s'], subset['umap_y'],
            s=12, alpha=0.8, label=f'cluster {label}'
        )

plt.xlabel("UMAP-1")
plt.ylabel("UMAP-2")
plt.title("UMAP + HDBSCAN Clustering")
plt.legend(box_to_anchor=(1.05, 1), loc='upper left', fontsize=8)
plt.tight_layout()
plt.show()
```

클러스터링 시각화 해서 확인하기

해당 파일 저장 함수!!!! 비교해야함!!!! 저장하려면 다시 불러야함!!!!

```
from scipy.cluster.hierarchy import dendrogram, linkage
from matplotlib import pyplot as plt
```

덴드로그램

```
link_model = linkage(list(result_df['vector']), 'ward') #ward 결합 기준으로

plt.figure(figsize=(15,9))
dendrogram(link_model,
            orientation='top',
            distance_sort='descending',
            show_leaf_counts=False)

plt.title("Dendrogram of Doc2Vec Embeddings")

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
dendrogram_filename = os.path.join(output_dir, 'dendrogram.png')
plt.savefig(dendrogram_filename)
plt.show()
print(f"Dendrogram saved to {dendrogram_filename}")
plt.close() # Close the plot to free memory
```

클러스터링 시각화해서 확인하기

```
from scipy.cluster.hierarchy import dendrogram, linkage
from matplotlib import pyplot as plt
```

덴드로그램

```
import ast
import numpy as np
from scipy.cluster.hierarchy import linkage

# 문자열 -> 실제 리스트
df['vector'] = df['vector'].apply(ast.literal_eval)

# (n_samples, n_features) 2차원 배열로 변환
X = np.vstack(df['vector'].values)

# 거리법 연결
link_model = linkage(X, method='ward')

import os

link_model = linkage(list(df['vector']), 'ward') #ward 결합 기준으로

# Individual plot is no longer saved separately
plt.figure(figsize=(15,9))
dendrogram(link_model,
            orientation='top',
            distance_sort='descending',
            show_leaf_counts=False)

plt.title("Dendrogram of ES Embeddings")

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
dendrogram_filename = os.path.join(output_dir, 'dendrogram.png')
plt.savefig(dendrogram_filename)
plt.show()
print(f"Dendrogram saved to {dendrogram_filename}")
plt.close() # Close the plot to free memory
```

PCA

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import os

# PCA 모델 설정
n_components = 2  # 시각화를 위한 2차원 공간으로 축소
pca = PCA(n_components=2, random_state=42)

# PCA-SBERT 변환된 PCA 적용
result_df['vector'] = numpy array 형태로 변환되어야 합니다.
pca_embedding = pca.fit_transform(list(df['vector']))

# 2차원 축소된 데이터를 DataFrame에 추가
df['pca_x'] = pca_embedding[:, 0]
df['pca_y'] = pca_embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(12, 10))
sns.scatterplot(
    x="pca_x",
    y="pca_y",
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title("PCA Projection of ES Embeddings", fontsize=16)
plt.xlabel("PCA Dimension 1", fontsize=12)
plt.ylabel("PCA Dimension 2", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
pca_filename = os.path.join(output_dir, 'pca_projection.png')
plt.savefig(pca_filename)
plt.show()
print(f"PCA plot saved to {pca_filename}")
plt.close() # Close the plot to free memory
```

```
import re
import pandas as pd
from sklearn import svm

# 데이터 로드
X = ...
y = ...

# SVM 모델 생성
model = svm.SVC()

# 모델 학습
model.fit(X, y)

# 예측
y_pred = model.predict(X)

# 정확도 계산
accuracy = (y == y_pred).sum() / len(y)

# 결과 출력
print(f"Accuracy: {accuracy}")
```

UMAP

```
import umap
import matplotlib.pyplot as plt
import seaborn as sns

umap_reducer = umap.UMAP(n_neighbors=15, min_dist=0.5)

embedding = umap_reducer.fit_transform(np.array(list(df['vector'])))

result_df['umap_s'] = embedding[:, 0]
result_df['umap_y'] = embedding[:, 1]

plt.figure(figsize=(10, 10))
sns.scatterplot(
    x="umap_s",
    y="umap_y",
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title("UMAP Projection of Doc2Vec Embeddings", fontsize=16)
plt.xlabel("UMAP Dimension 1", fontsize=12)
plt.ylabel("UMAP Dimension 2", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
umap_filename = os.path.join(output_dir, 'umap_projection.png')
plt.savefig(umap_filename)
plt.show()
print(f"UMAP plot saved to {umap_filename}")
plt.close() # Close the plot to free memory
```

t-SNE

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import os

# t-SNE 모델 설정
n_components = 2  # 시각화를 위한 2차원 공간으로 축소
perplexity, learning_rate, n_iter_max = 30, 200, 1000  # perplexity, learning_rate, n_iter_max은 TSNE의 하이퍼파라미터
random_state = 42  # 랜덤 시드 설정
link_model = TSNE(n_components=2, random_state=42, perplexity=30, learning_rate=200, n_iter_max=1000)

# Doc2Vec 결과에 t-SNE 적용
df['vector'] = link_model.fit_transform(np.array(list(df['vector'])))

# 결과 출력
df['time_s'] = time_embedding[:, 0]
df['time_y'] = time_embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(10, 10))
sns.scatterplot(
    x="time_s",
    y="time_y",
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title("t-SNE Projection of ES Embeddings", fontsize=16)
plt.xlabel("t-SNE Dimension 1", fontsize=12)
plt.ylabel("t-SNE Dimension 2", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
time_filename = os.path.join(output_dir, 'time_projection.png')
plt.savefig(time_filename)
plt.show()
print(f"t-SNE plot saved to {time_filename}")
plt.close() # Close the plot to free memory
```

Doc2Vec 학습시작기

```
from gensim.models.doc2vec import Doc2Vec
import numpy as np
import os

# 데이터 로드
X = ...
y = ...

# Doc2Vec 모델 생성
model = Doc2Vec(vector_size=100, min_count=1, min_count_per_word=1)

# 모델 학습
model.fit(X)

# 예측
y_pred = model.predict(X)

# 정확도 계산
accuracy = (y == y_pred).sum() / len(y)

# 결과 출력
print(f"Accuracy: {accuracy}")
```

PCA

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import os

# PCA 모델 설정
n_components = 2  # 시각화를 위한 2차원 공간으로 축소
pca = PCA(n_components=2, random_state=42)

# PCA-SBERT 변환된 PCA 적용
result_df['vector'] = numpy array 형태로 변환되어야 합니다.
pca_embedding = pca.fit_transform(list(df['vector']))

# 2차원 축소된 데이터를 DataFrame에 추가
df['pca_x'] = pca_embedding[:, 0]
df['pca_y'] = pca_embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(12, 10))
sns.scatterplot(
    x="pca_x",
    y="pca_y",
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title("PCA Projection of ES Embeddings", fontsize=16)
plt.xlabel("PCA Dimension 1", fontsize=12)
plt.ylabel("PCA Dimension 2", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
pca_filename = os.path.join(output_dir, 'pca_projection.png')
plt.savefig(pca_filename)
plt.show()
print(f"PCA plot saved to {pca_filename}")
plt.close() # Close the plot to free memory
```


| Appendix | 코드

```
import re

def clean_text(text):

    # URL 제거
    text = re.sub(r"http\S+", "", text)

    # HTML 제거
    text = re.sub(r"<.*>+", "", text)

    # @문자 제거
    text = re.sub(r"@[\w]+", "", text)

    # 여러 개 + 제거
    text = re.sub(r"(\+){1,2}", "==", text)

    # #### + ##
    text = re.sub(r"(\#){1,2}", "##", text)

    # !!!!! + !
    text = re.sub(r"(!){2,}", "!", text)

    # ????? + ?
    text = re.sub(r"(\?){2,}", "?", text)

    # 공백 정리
    text = re.sub(r"^\s+| ", "", text)

    return text.strip()
```

PCA

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns
import os

# PCA 결과 출력
# n_components는 시각화를 위한 2로 설정함(2D).
pca = PCA(n_components=2, random_state=42)

# Ko-SBERT Embedding PCA 적용
# result_df['vector']은 numpy array 형태로 반환되므로 2D.
pca_embedding = pca.fit_transform(list(df['vector']))

# 개별 Embedding 데이터를 dataframe에 추가
df['pca_x'] = pca_embedding[:, 0]
df['pca_y'] = pca_embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(12, 10))
sns.scatterplot(
    x='pca_x',
    y='pca_y',
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title('PCA Projection of GTE Embeddings', fontsize=16)
plt.xlabel('PCA Dimension 1', fontsize=12)
plt.ylabel('PCA Dimension 2', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
pca_filename = os.path.join(output_dir, 'pca_projection.png')
plt.savefig(pca_filename)
plt.show()
print(f"PCA plot saved to {pca_filename}")
plt.close() # Close the plot to free memory
```

클러스터링 시각화 해서 확인하기

```
from scipy.cluster.hierarchy import dendrogram, linkage
from matplotlib import pyplot as plt
```

덴드로그램

```
import os

link_model = linkage(list(df['vector']), 'ward') #ward 방법 기준으로

plt.figure(figsize=(15,8))
dendrogram(link_model,
            orientation='top',
            distance_sort='descending',
            show_leaf_counts=False)

plt.title('Dendrogram of Ko-SBERT Embeddings')

output_dir = './content/output_files'
os.makedirs(output_dir, exist_ok=True)
dendrogram_filename = os.path.join(output_dir, 'dendrogram.png')
plt.savefig(dendrogram_filename)
plt.show()
print(f"Dendrogram saved to {dendrogram_filename}")
plt.close() # Close the plot to free memory
```

▼ 모델 불러오기

```
11 import torch
import warnings
from sentence_transformers import SentenceTransformer

warnings.filterwarnings("ignore")

# GPU 사용 가능 여부 확인
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using device: {device}")

model = SentenceTransformer("BAAI/bge-m3", device=device)

# content 컬럼을 리스트로 변환
texts = df['content'].astype(str).tolist()

# 임베딩 생성
embeddings = model.encode(texts, normalize_embeddings=True)

# dataframe에 vector 원형 추가
df['vector'] = embeddings.tolist()
```

```
import torch
import warnings
import pandas as pd
from sentence_transformers import SentenceTransformer

warnings.filterwarnings("ignore")

# 장치에 CPU를 사용
device = 'cpu'
print(f"Using device: {device}")

# 모델 로드 (device를 'cpu'로 지정)
model = SentenceTransformer(
    'Alibaba-NLP/gte-multilingual-base',
    trust_remote_code=True,
    device=device
)

# 데이터 로딩 (None 데이터는 추가하고 추가하지는 않거나 빈 문자열을 포함할 수 있음)
# fillna("")를 추가하면 'noneType' 오류를 방지할 수 있으므로 2D로 텍스트 = df['merged_content'].fillna("").astype(str).tolist()

print(f"Total texts to process: {len(texts)}")

# 병렬로 생성 (GPU에서는 batch_size와 너무 작은 코어는 너무 작은 16-32 코어로 설정함(2D))
embeddings = model.encode(
    texts,
    batch_size=32,
    show_progress_bar=True,
    convert_to_numpy=True # CPU 작업 시 numpy 형식(2D)으로 변환함(2D)
)

# dataframe에 vector 원형 추가
df['vector'] = embeddings.tolist()

# 확인
print("Embedding completed")
print(df.head())
```

t-SNE

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np # numpy import 추가
import os

# t-SNE 결과 출력
# n_components는 시각화를 위한 2로 설정함(2D).
# perplexity, learning_rate, n_iter 등의 값에 따라서는 데이터에 따라 조정할 필요를 할 수 있음(2D).
# random_state를 설정하여 결과의 재현성을 확보함(2D).
tune_model = TSNE(n_components=2, random_state=42, perplexity=30, learning_rate=200, n_iter=1000)

# Decovar 확인용 t-SNE 적용
# result_df['vector']은 numpy array 형태로 반환되므로 2D.
# list(result_df['vector'])를 numpy 배열로 변환함(2D).
tune_embedding = tune_model.fit_transform(np.array(list(df['vector'])))

# 개별 Embedding 데이터를 dataframe에 추가
df['tune_x'] = tune_embedding[:, 0]
df['tune_y'] = tune_embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(12, 10))
sns.scatterplot(
    x='tune_x',
    y='tune_y',
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title('t-SNE Projection of GTE Embeddings', fontsize=16)
plt.xlabel('t-SNE Dimension 1', fontsize=12)
plt.ylabel('t-SNE Dimension 2', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
tune_filename = os.path.join(output_dir, 'tune_projection.png')
plt.savefig(tune_filename)
plt.show()
print(f"t-SNE plot saved to {tune_filename}")
plt.close() # Close the plot to free memory
```

▼ 모델 불러오기

```
11 import torch
import warnings
from sentence_transformers import SentenceTransformer

warnings.filterwarnings("ignore")

# GPU 사용 가능 여부 확인
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using device: {device}")

model = SentenceTransformer("BAAI/bge-m3", device=device)

# content 컬럼을 리스트로 변환
texts = df['content'].astype(str).tolist()

# 임베딩 생성
embeddings = model.encode(texts, normalize_embeddings=True)

# dataframe에 vector 원형 추가
df['vector'] = embeddings.tolist()

# 확인
df.head()
```

UMAP

```
import umap
import matplotlib.pyplot as plt
import seaborn as sns

umap_reducer = umap.UMAP(n_components=2, random_state=42)

embedding = umap_reducer.fit_transform(list(df['vector']))

df['umap_x'] = embedding[:, 0]
df['umap_y'] = embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(12, 10))
sns.scatterplot(
    x='umap_x',
    y='umap_y',
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title('UMAP Projection of GTE Embeddings', fontsize=16)
plt.xlabel('UMAP Dimension 1', fontsize=12)
plt.ylabel('UMAP Dimension 2', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
umap_filename = os.path.join(output_dir, 'umap_projection.png')
plt.savefig(umap_filename)
plt.show()
print(f"UMAP plot saved to {umap_filename}")
plt.close() # Close the plot to free memory
```

모델 불러오기

```
import torch
from sentence_transformers import SentenceTransformer

# GPU 사용 가능 여부 확인
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using device: {device}")

# 모델 로드
model = SentenceTransformer("jigan/ko-sbert-multitask", device=device)

# content 컬럼을 리스트로 변환
texts = df['content'].astype(str).tolist()

# 임베딩 생성
embeddings = model.encode(texts, batch_size=32, show_progress_bar=True)

# dataframe에 vector 원형 추가
df['vector'] = embeddings.tolist()

# 확인
df.head()
```

PCA

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns
import os

# PCA 결과 출력
# n_components는 시각화를 위한 2로 설정함(2D).
pca = PCA(n_components=2, random_state=42)

# Ko-SBERT Embedding PCA 적용
# result_df['vector']은 numpy array 형태로 반환되므로 2D.
pca_embedding = pca.fit_transform(list(df['vector']))

# 개별 Embedding 데이터를 dataframe에 추가
df['pca_x'] = pca_embedding[:, 0]
df['pca_y'] = pca_embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(12, 10))
sns.scatterplot(
    x='pca_x',
    y='pca_y',
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title('PCA Projection of Ko-SBERT Embeddings', fontsize=16)
plt.xlabel('PCA Dimension 1', fontsize=12)
plt.ylabel('PCA Dimension 2', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)

output_dir = './content/output_files'
os.makedirs(output_dir, exist_ok=True)
pca_filename = os.path.join(output_dir, 'pca_projection.png')
plt.savefig(pca_filename)
plt.show()
print(f"PCA plot saved to {pca_filename}")
plt.close() # Close the plot to free memory
```

데이터 전처리

```
import re

# URL 제거
text = re.sub(r"http\S+", "", text)

# HTML 제거
text = re.sub(r"<.*>+", "", text)

# @문자 제거
text = re.sub(r"@[\w]+", "", text)

# 여러 개 + 제거
text = re.sub(r"(\+){1,2}", "==", text)

# #### + ##
text = re.sub(r"(\#){1,2}", "##", text)

# !!!!! + !
text = re.sub(r"(!){2,}", "!", text)

# ????? + ?
text = re.sub(r"(\?){2,}", "?", text)

# 공백 정리
text = re.sub(r"^\s+| ", "", text)

return text.strip()
```

클러스터링 시각화해서 확인하기

```
from scipy.cluster.hierarchy import dendrogram, linkage
from matplotlib import pyplot as plt
```

덴드로그램

```
import os

link_model = linkage(list(df['vector']), 'ward') #ward 방법 기준으로

plt.figure(figsize=(15,8))
dendrogram(link_model,
            orientation='top',
            distance_sort='descending',
            show_leaf_counts=False)

plt.title('Dendrogram of GTE Embeddings')

output_dir = './output_files'
os.makedirs(output_dir, exist_ok=True)
dendrogram_filename = os.path.join(output_dir, 'dendrogram.png')
plt.savefig(dendrogram_filename)
plt.show()
print(f"Dendrogram saved to {dendrogram_filename}")
plt.close() # Close the plot to free memory
```

UMAP

```
import umap
import matplotlib.pyplot as plt
import seaborn as sns
import os

umap_reducer = umap.UMAP(n_components=2, random_state=42)

embedding = umap_reducer.fit_transform(list(df['vector']))

df['umap_x'] = embedding[:, 0]
df['umap_y'] = embedding[:, 1]

plt.figure(figsize=(12, 10))
sns.scatterplot(
    x='umap_x',
    y='umap_y',
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title('UMAP Projection of Ko-SBERT Embeddings', fontsize=16)
plt.xlabel('UMAP Dimension 1', fontsize=12)
plt.ylabel('UMAP Dimension 2', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)

output_dir = './content/output_files'
os.makedirs(output_dir, exist_ok=True)
umap_filename = os.path.join(output_dir, 'umap_projection.png')
plt.savefig(umap_filename)
plt.show()
print(f"UMAP plot saved to {umap_filename}")
plt.close() # Close the plot to free memory
```

t-SNE

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np # numpy import 추가
import os

# t-SNE 결과 출력
# n_components는 시각화를 위한 2로 설정함(2D).
# perplexity, learning_rate, n_iter 등의 값에 따라서는 데이터에 따라 조정할 필요를 할 수 있음(2D).
# random_state를 설정하여 결과의 재현성을 확보함(2D).
tune_model = TSNE(n_components=2, random_state=42, perplexity=30, learning_rate=200, n_iter=1000)

# Decovar 확인용 t-SNE 적용
# result_df['vector']은 numpy array 형태로 반환되므로 2D.
# list(result_df['vector'])를 numpy 배열로 변환함(2D).
tune_embedding = tune_model.fit_transform(np.array(list(df['vector'])))

# 개별 Embedding 데이터를 dataframe에 추가
df['tune_x'] = tune_embedding[:, 0]
df['tune_y'] = tune_embedding[:, 1]

# Individual plot is no longer saved separately
plt.figure(figsize=(12, 10))
sns.scatterplot(
    x='tune_x',
    y='tune_y',
    data=df,
    s=10, # 점 크기
    alpha=0.7 # 투명도
)

plt.title('t-SNE Projection of Ko-SBERT Embeddings', fontsize=16)
plt.xlabel('t-SNE Dimension 1', fontsize=12)
plt.ylabel('t-SNE Dimension 2', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)

output_dir = './content/output_files'
os.makedirs(output_dir, exist_ok=True)
tune_filename = os.path.join(output_dir, 'tune_projection.png')
plt.savefig(tune_filename)
plt.show()
print(f"t-SNE plot saved to {tune_filename}")
plt.close() # Close the plot to free memory
```


CJ제일제당. (2025, June 2). CJ제일제당, 'CJ더마켓' AI 대화형 자연어 검색 서비스 "Fai(파이)" 시작. CJ 뉴스룸.

<https://cjnews.cj.net/cj%EC%A0%9C%EC%9D%BC%EC%A0%9C%EB%8B%B9-cj%EB%8D%94%EB%A7%88%EC%BC%93-ai-%EB%8C%80%ED%99%94%ED%98%95-%EC%9E%90%EC%97%B0%EC%96%B4-%EA%B2%80%EC%83%89-%EC%84%9C%EB%B9%84%EC%8A%A4-fai/>

<https://www.mk.co.kr/news/special-edition/11425237>

<https://www.ascentkorea.com/why-is-search-engine-optimization-important-in-consumer-decision-journey/>

<https://www.dvn.ci/blog/ai-%EB%8F%84%EC%9E%85%EB%A5%A0%EC%9D%B4-%EA%B0%80%EC%9E%A5-%ED%99%9C%EB%B0%9C%ED%95%9C-%EA%B8%88%EC%9C%B5%EA%B6%8C-ai-%EC%96%B4%EB%96%BB%EA%B2%8C-%ED%99%9C%EC%9A%A9%ED%95%A0%EA%B9%8C-49000>

https://www.businesspost.co.kr/BP?command=article_view&num=193170

<https://biz.heraldcorp.com/article/10010034#:~:text=%EC%9D%B4%20%EB%8B%B4%EB%8B%B9%EC%9D%80%20%E2%80%9C%ED%94%84%EB%A1%9C%EA%B7%B8%EB%9E%98%EB%B0%8D%20%EC%96%B8%EC%96%B4%EB%A5%BC%20%EB%AA%A8%EB%A5%B4%EB%8A%94%20%EA%B5%AC%EC%84%B1%EC%9B%90%EB%8F%84%20%EB%8C%80%ED%99%94%ED%95%98%EB%93%AF,%EA%B5%AC%EB%A7%A4%20%EC%97%B0%EA%B2%B0%20%EB%B9%84%EC%9C%A8%EC%9D%B4%20%EC%A0%84%EB%85%84%20%EB%8C%80%EB%B9%84%20250%25%20%EC%83%81%EC%8A%B9%ED%95%98%EB%8A%94>

Lemon, K. N., & Verhoef, P. C. (2016). Understanding Customer Experience Throughout the Customer Journey. *Journal of Marketing*, 80(6), 69-96.

Suh, Youngjung. (2025). Discovering customer segments through interaction behaviors for home appliance business. *Journal of Big Data*. 12. 10.1186/s40537-025-01111-y.